

Master en Ciberseguridad en colaboración con Deloitte

Análisis Forense

Analizador de Registros del Sistema

Windows

TFM elaborado por: Pedro Rodríguez Treviño

Tutor/a de TFM: Daniel Maher López

Madrid. 30 de enero del 2026

RESUMEN

Durante el desarrollo de este Trabajo Fin de Máster, me propuse diseñar y construir una herramienta capaz de analizar los registros de eventos generados por sistemas operativos Windows, con el objetivo de facilitar la extracción, visualización y estudio de información relevante para el ámbito de la ciberseguridad y el análisis forense digital.

Los registros de eventos de Windows, almacenados en archivos con la extensión EVTX, contienen una gran cantidad de información sobre la actividad del sistema, el comportamiento de los usuarios, los procesos ejecutados y los posibles incidentes de seguridad. Sin embargo, su formato estructurado en XML y su almacenamiento binario dificultan su lectura directa, lo que complica su interpretación por parte de analistas o investigadores que no cuentan con herramientas especializadas.

A partir de esta problemática, decidí desarrollar una aplicación que ofreciera una interfaz gráfica intuitiva, capaz de abrir archivos EVTX, interpretar sus estructuras internas y mostrar su contenido de forma dinámica y personalizable. La herramienta fue implementada utilizando el lenguaje Python y el Framework PyQt6, lo que permitió combinar la flexibilidad del análisis programático con una interfaz moderna y eficiente. Además, se incorporó la librería Python-EVTX para la decodificación de los registros binarios, garantizando una lectura precisa y compatible con diferentes versiones del sistema operativo.

El principal objetivo de la aplicación es permitir al usuario explorar los eventos de manera dinámica, generando automáticamente las columnas y campos según la estructura interna de cada archivo de log. De este modo, la herramienta se adapta a distintos tipos de registros, ya sean del canal System, Security, Application u otros, sin requerir una configuración manual previa. Asimismo, el usuario puede aplicar filtros, exportar la información a formatos legibles y, en futuras versiones, integrar módulos adicionales orientados al análisis forense automatizado.

Este proyecto no solo busca ofrecer una solución técnica, sino también aportar una herramienta útil dentro del ecosistema de la ciberseguridad, en el que la capacidad de interpretar evidencias digitales de manera ágil y comprensible resulta esencial para la detección, mitigación y documentación de incidentes.

Palabras clave: Windows, Eventos de seguridad, Python

ABSTRACT

During the development of this Master's Thesis, I set out to design and build a tool capable of analyzing event logs generated by Windows operating systems, with the goal of facilitating the extraction, visualization, and analysis of information relevant to cybersecurity and digital forensics.

Windows event logs, stored in files with the EVTX extension, contain a wealth of information about system activity, user behavior, executed processes, and potential security incidents. However, their structured XML format and binary storage make them difficult to read directly, complicating their interpretation by analysts or researchers without specialized tools.

Based on this problem, I decided to develop an application that offered an intuitive graphical interface capable of opening EVTX files, interpreting their internal structures, and displaying their contents in a dynamic and customizable way. The tool was implemented using the Python language and the PyQt6 framework, which allowed it to combine the flexibility of programmatic analysis with a modern and efficient interface. Additionally, the Python-EVTX library was incorporated for decoding binary logs, ensuring accurate reading and compatibility with different operating system versions.

The main objective of the application is to allow the user to dynamically explore events, automatically generating columns and fields based on the internal structure of each log file. In this way, the tool adapts to different types of logs, whether from the System, Security, Application, or other channels, without requiring prior manual configuration. Furthermore, the user can apply filters, export information to readable formats, and, in future versions, integrate additional modules aimed at automated forensic analysis.

This project not only seeks to offer a technical solution, but also to provide a useful tool within the cybersecurity ecosystem, where the ability to interpret digital evidence in a flexible and understandable manner is essential for the detection, mitigation, and documentation of incidents.

Keywords: Windows, Security events, Python.

ÍNDICE

1.	<i>Introducción y antecedentes.....</i>	8
a.	Planteamiento del problema.	9
b.	Hipótesis y propuesta.	10
2.	<i>Marco teorico</i>	12
a.	Los eventos de seguridad en Windows.....	12
b.	Estructura de los eventos de seguridad y su valor tecnico.....	13
c.	Visor de eventos de Windows.....	13
d.	Herramientas forenses alternas	13
e.	El rol de XML en los eventos de Windows.....	14
f.	Procesos, auditorias e identificacion de actividades sospechosas.....	15
g.	Visualizacion y experiencia del usuario en herramientas forenses	15
3.	<i>La ciber seguridad en los registros</i>	17
a.	Importancia estrategica.....	17
b.	Respuesta ante incidentes	18
c.	Estructura de los registros y sus fundamentos teoricos	18
d.	Analisis de los registros.....	19
e.	Limitaciones teoricas.....	19
4.	<i>Objetivos.</i>	20
a.	Objetivos específicos.....	21

5.	<i>Materiales y métodos empleados.....</i>	<i>24</i>
a.	Entorno de desarrollo	24
b.	Lenguaje y librerías.....	25
c.	Diseño de la arquitectura	27
d.	Metodología de trabajo.	29
e.	Gestión de errores y pruebas.....	29
6.	<i>Resultados obtenidos.....</i>	<i>33</i>
a.	Descripción de herramienta.	33
b.	Evaluación funcional.....	36
c.	Áreas de mejora.....	37
7.	<i>Modelos utilizados.....</i>	<i>39</i>
a.	Modelo conceptual.....	39
b.	Modelo de datos.....	40
c.	Modelo de interfaz.....	40
d.	Modelo lógico de funcionamiento.....	42
8.	<i>Discusión y conclusiones.....</i>	<i>44</i>
a.	Interpretación de resultados.....	44
b.	Conclusiones generales.....	47
c.	Líneas futuras de desarrollo.....	48
9.	<i>Referencias bibliográficas.....</i>	<i>50</i>

10.	<i>Anexos.....</i>	<i>52</i>
a.	<i>Código fuente de la herramienta.</i>	<i>52</i>
1.	<i>Listado de gráficos.....</i>	<i>76</i>
2.	<i>Listado de tablas</i>	<i>77</i>

1. Introducción y antecedentes.

El crecimiento continuo de las infraestructuras tecnológicas ha provocado que la seguridad informática se convierta en una responsabilidad crítica dentro de organizaciones, empresas e instituciones. Cada equipo, servidor o servicio en funcionamiento genera una enorme cantidad de información, y entre esos datos destacan los registros de eventos, los cuales permiten comprender qué está ocurriendo en un sistema, quién lo está usando y bajo qué circunstancias. En el caso de Windows, estos registros toman la forma de los conocidos *Windows Event Logs*, que funcionan como una especie de “caja negra” del sistema operativo. Casi cualquier actividad relevante queda registrada allí de una manera u otra.

El problema, sin embargo, es que estos registros son extremadamente detallados, poco intuitivos y difíciles de analizar sin herramientas adecuadas. Aunque Windows proporciona su propio visor (Event Viewer), este presenta limitaciones importantes cuando se trabaja en análisis de incidentes, auditorías de seguridad o investigaciones forenses. Muchas veces resulta lento, poco flexible y carente de funciones avanzadas de filtrado o búsqueda que permitan identificar patrones sospechosos con facilidad. Además, la visualización del XML asociado a cada evento suele ser complicada y no siempre ofrece una navegación cómoda.

A partir de esta necesidad la de tener un visor más práctico, más ágil y enfocado al análisis de seguridad surgió este proyecto. La idea no era reinventar por completo la gestión de eventos, sino ofrecer una herramienta que hiciera más accesible y eficiente un proceso que, por naturaleza, es pesado y técnico. La motivación principal fue crear una solución que permitiera visualizar, explorar y comprender los eventos registrados por Windows sin necesidad de tener un conocimiento muy avanzado del sistema operativo o de herramientas forenses especializadas.

El desarrollo de esta herramienta también está estrechamente relacionado con la creciente importancia de la ciberseguridad a nivel global. En los últimos años, los ataques se han sofisticado, los métodos de intrusión son más variados y los analistas deben trabajar con tiempos más estrictos. Poder revisar logs de manera rápida y efectiva puede marcar la diferencia entre detectar un incidente a tiempo o pasar por alto un comportamiento anómalo que podría escalar más adelante. Por ello, contar con herramientas que faciliten el análisis se ha vuelto fundamental.

Otro antecedente importante es el aumento de investigadores, estudiantes y profesionales que buscan recursos accesibles para aprender análisis forense sin depender de herramientas costosas o complejas. Muchas plataformas avanzadas, aunque poderosas, no son accesibles para todos debido a sus costos o a sus requisitos técnicos. Esta situación impulsó la creación de una herramienta que, aun siendo sencilla, tuviera suficiente capacidad técnica para ser utilizada tanto

como apoyo de aprendizaje como en escenarios reales.

Desde el punto de vista del desarrollo, también había un interés claro en explorar las capacidades de PyQt6, un Framework potente para interfaces gráficas, que permite crear aplicaciones visualmente atractivas y con una lógica interna compleja. Integrar esta tecnología con librerías de análisis de eventos representó una oportunidad para aprender, practicar y demostrar habilidades técnicas vinculadas directamente a la ciberseguridad y al desarrollo de software.

Finalmente, este proyecto también se construyó como parte del proceso académico de formación, buscando unir conocimientos adquiridos en áreas como:

- Análisis forense digital
- Programación y arquitectura de software
- Diseño de interfaces
- Gestión de errores
- Sistematización de datos
- Herramientas de seguridad ofensiva y defensiva

La herramienta que se presenta en este trabajo surge, por lo tanto, como la respuesta a un conjunto de necesidades reales, tanto profesionales como académicas. Combina lo mejor del aprendizaje práctico con una aplicación clara y concreta: facilitar el análisis de eventos del sistema Windows de manera rápida, ordenada y visualmente comprensible. A largo plazo, este proyecto busca demostrar que es posible desarrollar soluciones útiles y completamente funcionales con recursos accesibles, poniendo la tecnología al servicio de la investigación y la seguridad.

a. Planteamiento del problema.

El análisis de eventos del sistema operativo Windows es una de las tareas más fundamentales dentro de la informática forense y la seguridad cibernética. Sin embargo, esta tarea presenta una serie de desafíos importantes que dificultan su realización, especialmente cuando no se dispone de herramientas adecuadas o que requieren de mucha lectura a documentación cuando hay casos que requieren una lectura rápida que no comprometa los resultados que se pueden obtener de dicha lectura.

Uno de los principales problemas es la complejidad inherente de los registros. Cada evento genera una estructura extensiva de información, muchas veces en formato XML, que no está diseñada para ser leída fácilmente por un ser humano. Además, los logs contienen una enorme cantidad de detalles técnicos, códigos, identificadores, atributos y parámetros que requieren interpretación. Incluso para un analista experimentado, revisar estos datos puede ser una tarea lenta y agotadora.

Otro problema significativo es que Windows Event Viewer, la herramienta integrada por defecto en el sistema no siempre es suficiente. Aunque permite visualizar los eventos, su interfaz puede resultar confusa y limitada. Filtrar grandes cantidades de eventos, buscar coincidencias específicas o navegar entre estructuras XML profundas puede volverse un proceso tedioso. Además, cuando se manejan archivos EVTX grandes, el visor nativo tiende a volverse lento o incluso a congelarse.

Por otro lado, en el contexto del análisis forense, el tiempo es un factor crucial. Cuando ocurre un incidente de seguridad, el analista necesita identificar rápidamente qué sucedió: qué proceso se ejecutó, qué usuario inició sesión, desde qué dirección IP, qué servicios fallaron, qué archivos se modificaron, etc. Si la herramienta dificulta la búsqueda de información relevante, el proceso de investigación puede retrasarse y afectar la toma de decisiones.

Finalmente, existe también una necesidad educativa. Muchos estudiantes y futuros analistas requieren herramientas accesibles que les permitan practicar con logs reales sin tener que recurrir a software costoso o herramientas cerradas. La ausencia de herramientas ligeras y adaptadas a un nivel de entrada genera una barrera que puede dificultar la formación en estas áreas.

Por todo esto, surgió el problema principal que esta tesis busca abordar: la falta de una herramienta práctica, ligera y accesible que permita analizar eventos de Windows de manera rápida, intuitiva y completa, evitando las limitaciones del visor nativo y facilitando tanto la exploración como la interpretación de los datos.

b. Hipótesis y propuesta.

Se plantea como hipótesis principal que el desarrollo de una herramienta de análisis de archivos EVTX basada en tecnologías abiertas, particularmente en Python y PyQt6, puede ofrecer un entorno eficaz, adaptable y de fácil uso para la interpretación de registros de eventos de Windows, sin depender de aplicaciones comerciales o privilegios administrativos avanzados.

Esta hipótesis parte de la idea de que el uso de bibliotecas especializadas, como Python-EVTX, en combinación con una interfaz gráfica moderna, puede simplificar significativamente el proceso de exploración y filtrado de datos de eventos, contribuyendo tanto al ámbito educativo como al profesional.

La propuesta de este trabajo consiste en el diseño y desarrollo de una aplicación de escritorio que permita la lectura, análisis y visualización de archivos EVTX provenientes de sistemas Windows.

El proyecto contempla los siguientes ejes principales:

- Interfaz gráfica de usuario construida usando PyQt6 para ofrecer una experiencia visual intuitiva y moderna, con soporte para filtrado y búsqueda dinámica de registros.
- Módulo de análisis de eventos basado en la biblioteca Python-EVTX, en la que se extraen los datos contenidos en los registros, incluyendo la identificación automática de campos XML (encabezados dinámicos).
- Visualización y exportación de datos mediante tablas interactivas que permitan visualizar los eventos y exportarlos a formatos abiertos, siendo estos CSV para su posterior análisis.

De esta manera, la herramienta busca combinar accesibilidad y funcionalidad, proporcionando una alternativa educativa y de investigación válida para la comunidad de ciberseguridad y análisis forense digital.

2. Marco teórico

El análisis de registros de eventos de sistemas operativos constituye una de las disciplinas fundamentales dentro de la ciberseguridad, la informática forense y la administración de sistemas. En el caso particular de Windows, los *Event Logs* representan una fuente de información crítica para comprender el comportamiento del sistema, identificar incidentes de seguridad, analizar fallos y reconstruir actividades realizadas por usuarios y procesos. El objetivo de este marco teórico es proporcionar una base conceptual sólida que contextualice el proyecto, establezca criterios técnicos, explique la importancia del análisis de eventos y presente los elementos clave que sustentan la construcción de la herramienta desarrollada.

a. Los eventos de seguridad en Windows

Los sistemas Windows registran prácticamente todas las acciones relevantes que ocurren en el equipo: accesos, inicios de sesión, instalación de software, fallos del sistema, uso de servicios, modificaciones de configuraciones, ejecución de procesos, entre otros. Todos estos registros quedan almacenados en un formato conocido como **EVTX**, el cual constituye la columna vertebral del análisis forense en entornos Windows.

Para un analista forense, los eventos de seguridad son componentes fundamentales de cualquier caso, esto debido a que dichos eventos permiten la reconstrucción de incidentes desde el momento 0, de forma que se puedan identificar ataques, detectar comportamientos anómalos dentro de los propios registros y a partir de dichos comportamientos, rastrear accesos o procesos indebidos; Los eventos de seguridad y su posterior análisis son clave dentro de cualquier estrategia de seguridad o investigación.

Es importante destacar que los registros de Windows están separados por categorías que agrupan diferentes tipos de información, de los cuales destacan:

- Seguridad: Eventos que detallan los componentes más importantes de un análisis forense, siendo estos los eventos de inicio y cierre de sesión, así como lo que puede llevar a la escalada de privilegios, usurpación de identidad, intentos fallidos de autenticación (y, por ende, ataques de fuerza bruta) y auditorias de permisos
- Sistema: Enfocados en el registro de eventos relacionados con el Kernel del sistema y servicios internos, ya sea la administración de drivers, actualizaciones, fallos de servicios y apagados inesperados
- Aplicaciones: Enfocado a eventos generados por programas instalados o servicios específicos

b. Estructura de los eventos de seguridad y su valor técnico

El formato EVTX fue introducido por Microsoft como sustituto del antiguo formato EVT. Se caracteriza por ser un formato binario complejo basado en fragmentos estructurados que contienen representaciones de eventos en XML que contienen metadatos asociados a procesos, tiempos, usuarios y servicios, además de que contiene datos específicos del evento y un cuerpo XML con la estructura completa

Las ventajas que tiene este formato se basan en que no dependen de texto plano, reduciendo corrupción de archivos y mejorando la eficiencia al permitir almacenar miles de eventos por archivo y facilita su recuperación de datos incluso en archivos parcialmente dañados; No obstante, este formato no está exento de dificultades a la hora de utilizarlo, dígame su complejidad interna que limita la creación de parsers caseros, además de que para esto requiere el uso de librerías especializadas y pueden llevar a una complejidad de requisitos, también cabe destacar que el formato XML puede contener estructuras anidadas difíciles de interpretar a simple vista, también esto puede causar inconsistencias entre identificadores.

c. Visor de eventos de Windows

Windows Event Viewer es la herramienta nativa de Windows diseñada para visualizar los logs, pero su arquitectura presenta limitaciones reconocidas por analistas.

Limitaciones principales:

- El motor de búsqueda es poco flexible
- Los filtros avanzados son difíciles de utilizar
- Navegar el XML es incómodo y no permite resaltar coincidencias
- El rendimiento no es óptimo con archivos grandes
- La interfaz no facilita el análisis manual intensivo
- No muestra relaciones ni dependencias entre eventos
- No permite analizar varios archivos a la vez

La combinación de estas limitaciones hace que, aunque Event Viewer sea útil para visualizaciones rápidas, resulte insuficiente para análisis forense profundo y muchos analistas opten por recurrir a herramientas externas.

d. Herramientas forenses alternas

El ecosistema de ciber seguridad incluye herramientas avanzadas diseñadas para manejar logs en grandes volúmenes, conocidas como SIEM (Security Información and Event

Management). Entre ellas destacan Splunk, ELK (Elasticsearch, Logstash, Kibana) y Graylog.

Ventajas de un SIEM:

- Análisis distribuido
- Correlación de eventos
- Dashboards
- Búsqueda avanzada
- Alertas en tiempo real
- Ingestión de logs masivos

Desventajas para el contexto de este proyecto:

- Requieren infraestructura compleja
- Consumen grandes recursos
- Tienen costos elevados
- No son herramientas portátiles
- No están pensadas para abrir archivos EVTX locales para un análisis directo.

Por otro lado, existen herramientas forenses ligeras como:

- Chainsaw
- EvtxECmd
- DeepBlueCLI

pero la mayoría se basan en línea de comandos, carecen de interfaz gráfica y están orientadas a análisis automático, cosa que, a mi parecer, es algo que debe de tomarse con mucho cuidado, en cualquier caso, esto al depender de un análisis automático que no siempre es fiable y puede cometer errores.

Eso deja un hueco importante: una herramienta liviana, gráfica, portable y fácil de entender para análisis humano, siendo esto un espacio para mi propuesta.

e. El rol de XML en los eventos de Windows

Uno de los elementos más poderosos del formato EVTX es que los eventos están estructurados en XML, dato que será más importante al momento de realizar el análisis de como estructurar la herramienta para manejar este formato.

Cada elemento XML representa:

- Propiedades básicas
- Valores clave
- Parámetros adicionales
- Contexto del evento

- Metadatos útiles para el análisis

Sin embargo, este XML suele ser bastante extenso, con:

- Jerarquías profundas
- Múltiples atributos
- Espacios de nombres (namespaces)
- Campos no intuitivos
- Información redundante

Por eso, interpretarlo directamente desde Event Viewer puede ser difícil. Una herramienta que presenta ese XML en un árbol expandible y visualmente ordenado aporta una ventaja sustancial al análisis.

f. Procesos, auditorías e identificación de actividades sospechosas

Muchos ataques y comportamientos maliciosos dejan rastros claros en los logs de Windows, si se sabe dónde buscar. Por ejemplo:

- Intentos fallidos de inicio de sesión (4625): Suelen indicar ataques de fuerza bruta o contraseñas incorrectas repetidas.
- Creación de procesos (4688): Esencial para detectar malware, scripts sospechosos o ejecución de PowerShell.
- Asignación de privilegios especiales (4672): Alerta sobre elevación de privilegios.
- Borrado de logs (1102): Casi siempre actividad maliciosa o administrativa crítica.
- Ejecución de comandos en PowerShell (4104): Puede revelar la ejecución de scripts maliciosos o descargas de payloads.

Estos conceptos utilizados a la hora de realizar auditorías y análisis de casos son importantes recordarlos ya que suelen ser los eventos que resaltan a la vista al momento de crear un caso.

g. Visualización y experiencia del usuario en herramientas forenses

La interfaz gráfica es una parte clave del análisis. En ciberseguridad, donde se manejan grandes volúmenes de información técnica, la visualización adecuada puede significar que se puede realizar una detección de incidentes a tiempo, evitando errores de interpretación y agilizando el análisis, de forma que se facilitan los procesos que conllevan investigaciones

prolongadas

Los principios de UX aplicados que deberían incluirse en una herramienta enfocada en la simplicidad son:

- Diseño limpio
- Modo oscuro
- Agrupación de elementos
- Iconografía clara
- Uso de tablas dinámicas

Este marco establece los fundamentos esenciales sobre los cuales se construye el proyecto, los cuales son los eventos de Windows, la razón de su importancia dentro del proyecto, cómo funcionan las herramientas existentes, cuáles son sus limitaciones y qué papel cumplen tecnologías como XML, PyQt6 y EVTX en el análisis forense y en el desarrollo de mi propuesta, también cabe destacar que el desarrollo del proyecto no va solamente del lado técnico, sino que también tiene enfoque dentro del ecosistema actual de seguridad informática.

3. La ciber seguridad en los registros

En cualquier sistema informático moderno, los registros o logs se han convertido en una de las fuentes de información más valiosas para comprender el comportamiento interno de un equipo, detectar actividad anómala, reconstruir incidentes e incluso demostrar el cumplimiento de normativas de seguridad. En términos simples, un log es una evidencia textual o estructurada que documenta lo que está ocurriendo dentro de un sistema. Sin embargo, en el contexto de ciberseguridad, su importancia va mucho más allá de un simple historial: los logs son uno de los pilares fundamentales que permiten la detección temprana de amenazas, el análisis forense digital y la respuesta ante incidentes.

Dentro del ecosistema Windows, esta relevancia es aún mayor ya que el sistema genera un volumen inmenso de registros que abarcan eventos relacionados con autenticaciones, accesos a recursos, ejecución de procesos, cambios en la configuración, errores del sistema, operaciones administrativas, políticas avanzadas y prácticamente cualquier acción que implique un riesgo potencial. Comprender cómo se generan, estructuran y utilizan estos registros se vuelve esencial para cualquier profesional dedicado a la seguridad informática o a la investigación de incidentes.

a. Importancia estratégica

La importancia del logging se puede analizar desde varias perspectivas: técnica, operativa, legal y de gestión de riesgos, en todas ellas los registros cumplen funciones específicas, pero interconectadas, de manera que se puede construir un caso con evidencias de distintos registros o logs.

En seguridad, uno de los principios más importantes es que “no se puede proteger lo que no se puede ver”, los logs son, precisamente, esa ventana a lo que sucede dentro del sistema. Proporcionan trazabilidad, contexto y cronología a cualquier acción realizada dentro del sistema analizado, elementos sin los cuales sería imposible entender un incidente o diagnosticar la raíz de un problema.

Muchos ataques no generan fallos visibles para el usuario, pero sí dejan rastros en los registros, siendo los más comunes los intentos de inicio de sesión fallidos, escalamiento de privilegios, creación de tareas programadas, manipulación del registro de Windows, ejecución de scripts desconocidos o fallos inesperados en servicios clave, todo esto queda almacenado en forma de eventos en distintos canales/ficheros EVTX. Por esto, las plataformas SIEM (Security Information and Event Management) dependen directamente de los logs para detectar comportamientos sospechosos en tiempo real, y es por la misma razón que estos ficheros son

fundamentales para el programa a desarrollar.

b. Respuesta ante incidentes

Cuando un incidente ya ha ocurrido, los logs son la herramienta principal para reconstruir los hechos:

- Qué usuario estuvo involucrado
- Qué proceso se ejecutó
- Fecha del acceso indebido
- Qué dirección IP se utilizó
- Qué cambios se hicieron al sistema
- Qué errores aparecieron previamente

Un análisis forense sin logs es casi como intentar resolver un crimen sin testigos, sin cámaras y sin evidencia física, en otras palabras, te encontrarías en total desconocimiento de la situación.

c. Estructura de los registros y sus fundamentos teóricos

Para entender su relevancia en ciberseguridad, es necesario comprender su estructura interna. En esencia, un log es un mensaje estructurado compuesto por varios campos que describen un suceso. Aunque varía según el sistema, generalmente contiene los campos más relevantes para cualquier tipo de evento:

- Timestamp
- Fuente (source)
- ID del evento
- Tipo/severidad
- Usuario involucrado
- Equipo o proceso asociado
- Mensaje descriptivo

En Windows, estos eventos se almacenan en archivos EVTX, un formato binario diseñado para permitir rapidez de consulta, integridad y eficiencia, incluso en sistemas con cientos de miles de eventos diarios, como ya se había mencionado, la compatibilidad de este formato con el proyecto es esencial para la planeación de librerías a utilizar y funcionamiento de la herramienta propuesta.

d. Análisis de los registros

El análisis de logs no se limita a leer mensajes, sino que implica interpretar señales, patrones y anomalías, apoyándose en principios teóricos, entre ellas destaca la identificación de patrones conocidos de ataques basados en eventos específicos sin dejar de lado el contexto de la situación.

Por ejemplo:

- Un usuario administrativo iniciando sesión a las 3 AM
- Un proceso ejecutándose desde una ruta temporal
- Muchos eventos 4625 seguidos
- Cambios en políticas sin explicación

El comportamiento anómalo es más revelador que un evento aislado, por ende, combinar múltiples eventos en secuencia permite reconstruir narrativas, siendo el caso de que hay fallos repetidos de intento de acceso, lo que puede conllevar a escalamiento, ejecución y finalmente , exfiltración. Esto es básicamente la base filosófica detrás de la mayoría de los sistemas modernos de detección de amenazas.

e. Limitaciones teóricas

A pesar de que se detallan estrategias y patrones, no es tan simple, el análisis de registros conlleva desafíos importantes, tal es el caso de:

- Exceso de volumen (millones de eventos en ambientes corporativos)
- Falsos positivos
- Eventos redundantes
- Correlación difícil
- Necesidad de herramientas especializadas
- Riesgo de manipulación por parte de atacantes

Incluso así, las ventajas superan ampliamente las limitaciones.

El análisis de logs no es una práctica secundaria, sino una disciplina central dentro de la ciberseguridad. Las organizaciones que adoptan estrategias basadas en análisis de registros mejoran significativamente su capacidad de detectar ataques, responder incidentes y garantizar la integridad del sistema. Desde los modelos normativos hasta las implementaciones modernas como Windows Event Log, todo apunta hacia la importancia crítica de los registros como mecanismo de defensa.

4. Objetivos.

El desarrollo de esta herramienta no surgió de manera improvisada, sino que fue el resultado de una necesidad concreta y recurrente dentro del ámbito del análisis de seguridad y la informática forense: la dificultad para interpretar los registros de eventos generados por sistemas Windows. A partir de esa necesidad surgió la idea de diseñar un sistema capaz de hacer que estos datos fueran más accesibles, comprensibles y útiles utilizando una herramienta que permita una interpretación sencilla y rápida de los datos presentes en los archivos de eventos de seguridad de Windows.

En esta sección explico con detalle los objetivos que guiaron el proyecto, no solo como una lista de metas técnicas, sino como un conjunto de propósitos relacionados con la experiencia del usuario, la funcionalidad del software, la claridad del análisis y el aprendizaje durante el desarrollo. El objetivo no fue únicamente construir un programa, sino construir una herramienta que resolviera un problema real.

El objetivo general del proyecto fue desarrollar una herramienta completa, funcional y visualmente clara para la lectura, visualización, exploración y análisis de eventos del sistema operativo Windows, tanto desde archivos de eventos como desde los canales de eventos propios del sistema.

Pero este objetivo no se limita a crear un visor más del estilo de Event Viewer. El propósito más profundo fue demostrar que es posible:

- Transformar información técnica y compleja en algo navegable
- Facilitar el trabajo de investigación forense
- Mejorar la experiencia del usuario frente a herramientas tradicionales
- Promover una aproximación más accesible a la interpretación de registros

El objetivo general también incluye la intención de aprender, analizar y aplicar buenas prácticas de desarrollo dentro del contexto de una herramienta real, no simplemente un prototipo académico. La interfaz, la arquitectura, el procesamiento y la interpretación de datos debían funcionar de manera integrada, natural y eficiente.

Otra parte importante del objetivo general es que la herramienta fuera útil tanto para:

- Estudiantes
- Analistas principiantes
- Personal de TI
- Profesionales que necesiten un visor ligero y rápido

No se trató solo de construir una aplicación, sino que también se buscó desarrollar una

solución que conectara la teoría con la práctica y que pudiera servir como punto de partida para proyectos más avanzados, esto mediante la colaboración de interesados en el proyecto mediante el uso de GitHub para poder crear una comunidad enfocada en la mejora continua de la herramienta propuesta para cumplir con los objetivos planteados.

a. Objetivos específicos.

Para alcanzar el objetivo general, se establecen los siguientes objetivos específicos:

1. Diseñar una interfaz gráfica clara, moderna y funcional

Uno de los objetivos centrales fue construir una interfaz suficientemente intuitiva para que cualquier usuario pudiera entenderla sin necesidad de consultar documentación externa. Esto implicó:

- Estructurar visualmente la herramienta en dos paneles principales: uno para la vista general (tabla) y otro para la vista detallada (XML).
- Crear una barra superior de control que agrupa las funciones esenciales del programa: cargar archivos, cargar canales, buscar, filtrar y exportar resultados.
- Utilizar hojas de estilo (QSS) para proporcionar una experiencia visual más agradable y coherente.

La interfaz responde a la idea de que un buen análisis forense no depende solo de la información disponible, sino también de la manera en la que se presenta esa información.

2. Implementar un sistema de carga y procesamiento de registros eficiente

Dado que los archivos EVTX pueden contener miles o incluso decenas de miles de eventos, otro objetivo importante fue asegurar que la herramienta pudiera procesarlos sin bloquearse ni afectar la experiencia del usuario. Para ello se planteó:

- Implementar hilos de ejecución (threads) para la carga de archivos.
- Garantizar la compatibilidad tanto con archivos externos como con canales nativos del sistema (Security, System, Application).
- Normalizar los campos extraídos del XML para asegurar consistencia en los nombres y valores.

Este objetivo buscaba que la carga fuera lo más fluida posible y que la herramienta siguiera siendo usable incluso con archivos grandes.

3. Desarrollar capacidades avanzadas de filtrado y búsqueda

Otro de los puntos clave del proyecto fue permitir que los usuarios encontraran eventos específicos o patrones relevantes dentro de grandes volúmenes de datos. Por ello, se plantearon los siguientes subobjetivos:

- Incluir una búsqueda global, capaz de encontrar coincidencias tanto en la tabla como en el XML.
- Añadir un sistema de filtros avanzados que permitiera búsquedas complejas mediante operadores lógicos, rangos de tiempo, coincidencias parciales y expresiones regulares.
- Permitir filtrar por columna mediante menús contextuales.

Estas funciones convierten la herramienta en algo más que un simple visor: permiten realizar un análisis real y detallado.

4. Proveer una visualización XML estructurada y fácil de navegar

El XML es el formato original de los registros de eventos de Windows. Entenderlo correctamente es fundamental para un análisis profundo. Por ello, uno de los objetivos fue construir un visor XML:

- Organizado en forma de árbol jerárquico.
- Capaz de expandir nodos dinámicamente.
- Relacionado directamente con el evento seleccionado en la tabla.

Este diseño permite a los usuarios interpretar el contenido exacto del evento, tal como aparece en el archivo EVTX, sin necesidad de un análisis manual o de herramientas externas.

5. Añadir resaltado de eventos críticos o sospechosos

Durante el desarrollo surgió la idea de integrar un objetivo adicional: identificar automáticamente eventos que suelen tener relevancia en investigaciones forenses o incidentes de seguridad.

El objetivo incluyó:

- Resaltar eventos como 4625 (fallo de autenticación), 4688 (creación de proceso), 4672 (privilegios especiales), 1102 (borrado del registro), entre otros.
- Permitir que el analista detectara patrones de interés rápidamente.

Este objetivo ayuda a que la herramienta no solo muestre información, sino que también “guíe” al analista hacia eventos importantes.

6. Facilitar la exportación de resultados

Finalmente, se estableció como objetivo que la herramienta permitiera

exportar la tabla de eventos filtrada para su uso en reportes o análisis externos.

Esto incluye:

- Exportación directa a formato CSV.
- Preservación del estado actual de los datos (columnas activas, filtros aplicados, ordenamiento).

Este objetivo busca apoyar el trabajo documental del analista, especialmente cuando es necesario presentar hallazgos.

En conjunto, estos objetivos específicos definieron el alcance y la dirección del proyecto. Cada uno de ellos contribuyó a construir una herramienta que no solo funciona bien desde el punto de vista técnico, sino que también resulta útil para fines prácticos y educativos. El proyecto logró integrar esta serie de objetivos en un sistema cohesivo, funcional y flexible, demostrando que es posible desarrollar herramientas de análisis forense accesibles pero potentes a partir de tecnologías disponibles y de código abierto.

Además, el enfoque modular del proyecto busca que la herramienta sea fácilmente ampliable o reutilizable, fomentando su uso en actividades de formación, investigación y práctica profesional dentro del ámbito de la ciberseguridad y el análisis forense digital (Jones et al., 2020).

El desarrollo de una aplicación propia no solo representa un ejercicio de ingeniería de software, sino también una aportación al ecosistema educativo y de investigación.

5. Materiales y métodos empleados.

El desarrollo de la herramienta requirió una combinación cuidadosa de tecnologías de software, técnicas de programación, metodologías de trabajo y enfoques propios del análisis forense digital. La complejidad del proyecto no radicó únicamente en escribir código, sino en integrar diferentes componentes que permitieran transformar los registros de eventos de Windows normalmente difíciles de interpretar en información estructurada, navegable y visualmente comprensible.

En esta sección se describen de forma detallada todos los materiales utilizados, así como los métodos empleados para llevar a cabo el diseño, la implementación y la validación de la herramienta. El objetivo es proporcionar una visión completa del proceso, permitiendo entender las decisiones técnicas adoptadas, los fundamentos que las motivaron y la manera en que cada elemento contribuyó al resultado final.

a. Entorno de desarrollo

El entorno de desarrollo seleccionado fue clave para garantizar la estabilidad, portabilidad y modularidad de la herramienta. Se trabajó principalmente en un equipo con sistema operativo Windows, lo cual era indispensable considerando que los logs EVTX y los canales de eventos del sistema son tecnologías nativas de Windows. Aunque el proyecto podría desarrollarse en otros sistemas mediante entornos virtualizados, trabajar directamente sobre Windows simplificó enormemente las pruebas y la integración con las APIs del sistema, así como la obtención de los archivos de eventos de seguridad utilizados en las pruebas, así como acceso a los canales de eventos del propio sistema.

El entorno utilizado para el desarrollo del programa incluía:

- Sistema operativo: **Windows 10 / 11**, actualizaciones recientes.
- Editor principal: **Visual Studio Code**, con extensiones para Python, Qt, formateo automático y Linting.
- Intérprete: **Python 3.11**, elegido por su compatibilidad con PyQt6 y librerías de análisis.
- Paquetes instalados:
 - PyQt6
 - python-evtx
 - pywin32 (para lectura de canales del sistema)
 - lxml (para manejo de XML cuando fue necesario)
 - logging y módulos estándar de Python

Al momento de decidir cuál sería mi entorno de desarrollo, lo que me motivó a elegir al que describí fue que es un entorno ligero y ampliamente utilizado previamente por mí en proyectos pasados, evitando soluciones más pesadas o con demasiada configuración como PyCharm o entornos virtualizados complejos, aunado a esto, otros factores que contribuyeron a esta decisión fueron:

- La facilidad para instalar dependencias que fueran necesarias para el desarrollo mediante el Marketplace de Visual Studio Code y la terminal con PIP
- El alto nivel de personalización con el que cuenta Visual Studio Code para un desarrollo más cómodo
- La integración nativa con el sistema Windows, permitiendo acceso sencillo a canales y ficheros de eventos
- La posibilidad de trabajar con archivos EVTX reales en tiempo real, siendo estos los canales principales del sistema

Además, Visual Studio Code proporcionó un entorno cómodo para manejar múltiples archivos, abrir ventanas simultáneas y visualizar los resultados rápidamente.

b. Lenguaje y librerías.

El lenguaje elegido fue Python, no solo por su popularidad, sino porque ofrece una combinación ideal de:

- Rapidez de desarrollo debido a múltiples características del lenguaje que facilitan el desarrollo de programas e integración de características
- Disponibilidad de librerías para análisis forense que facilitan la creación del programa
- Integración con APIs de Windows para un análisis más sencillo y acceso a canales de eventos del propio sistema
- Capacidades gráficas mediante PyQt6
- Sintaxis clara que facilita el mantenimiento del código, así como su entendimiento para futuras iteraciones por parte de alguien ajeno a mí

PyQt6 fue seleccionado como Framework principal de la interfaz gráfica del usuario por las siguientes razones:

- Ofrece widgets avanzados sin necesidad de programar elementos visuales desde cero, facilitando y agilizando el desarrollo del componente visual del programa
- Permite crear tablas, árboles XML y barras de menú de manera rápida
- Tiene soporte nativo para eventos, hilos y señales

- Proporciona estilos CSS mediante QSS que permiten lograr interfaces modernas, siendo este el caso de componentes como:
 - QMainWindow para la estructura base,
 - QTableWidget para los eventos,
 - QTreeWidget para el XML,
 - QHBoxLayout y QVBoxLayout para la composición visual,
 - QFileDialog para cargar EVTX,
 - QMessageBox para manejo de errores y alertas.

La combinación de estos widgets permitió lograr una interfaz clara, profesional y fácil de navegar, siguiendo lo planteado en los objetivos del proyecto.

La librería python-evtx fue fundamental para la normalización de archivos de eventos, esto debido a que esta librería permite:

- Abrir archivos de eventos de seguridad
- Navegar por los registros de cada evento
- Extraer estructuras XML completas de cada evento, de manera que puedan ser interpretadas y formateadas en futuros procesos

Esta librería facilitó enormemente el análisis interno de los eventos, ya que replicar desde cero un parser EVTX sería inviable en tiempo y complejidad.

Para leer los eventos directamente desde los canales del sistema Windows (como *Security*, *System* o *Application*), se utilizó la librería pywin32, que permite conectarse al Event Log API de Windows y obtener eventos en tiempo real, esto sin necesidad de especificar la ruta del archivo, además de que datos más reales no se pueden usar que los propios del sistema, esta decisión de agregar un conducto directo a los canales del sistema se basó en el objetivo de que la herramienta sea sencilla de usar.

Esto permitió que la herramienta fuera capaz de:

- Leer eventos sin necesidad de exportar un EVTX externo
- Obtener información actualizada directamente del sistema
- Realizar análisis forense en vivo.

El correcto funcionamiento de la herramienta requirió también de la integración de diversas bibliotecas y paquetes distintas a las ya mencionadas, tanto estándar como de terceros. A continuación, se detallan las otras librerías fundamentales para el correcto funcionamiento del

programa:

- i. *xml.etree.ElementTree*: módulo estándar utilizado para parsear los eventos en formato XML y extraer los campos relevantes, como la fecha, el origen, el identificador y los mensajes asociados.
- ii. *csv*: módulo estándar utilizado para la exportación de los resultados a archivos delimitados por comas, permitiendo su análisis posterior mediante hojas de cálculo o herramientas externas.
- iii. *ctypes*: utilizado para solicitar privilegios de administrador y relanzar la aplicación con permisos elevados, requisito necesario para acceder a ciertos canales protegidos del sistema.
- iv. *os*, *tempfile* y *shutil*: empleados para la gestión de rutas de archivos, la creación de copias temporales y la manipulación segura de directorios del sistema.

Esta combinación de bibliotecas permitió crear una herramienta integral, capaz de interactuar directamente con el sistema operativo, procesar los datos y presentarlos de forma visual, todo dentro de un mismo entorno de desarrollo.

c. Diseño de la arquitectura.

i. Visión general

La herramienta fue construida sobre una arquitectura modular, cuyo objetivo principal fue permitir que cada componente tuviera una responsabilidad clara. Esto no solo facilita el mantenimiento del código, sino que también permite agregar funcionalidades nuevas sin romper el funcionamiento existente.

De esta manera, el sistema se compone de tres niveles bien definidos:

- Capa de presentación: encargada de la interacción directa con el usuario, construida sobre la biblioteca PyQt6.
- Capa de control o lógica: responsable de coordinar las acciones entre la interfaz y el acceso a los datos.
- Capa de datos o backend: encargada de la comunicación directa con el sistema operativo y de la lectura de los registros del sistema.

Esta división facilita no solo el mantenimiento del código, sino también su escalabilidad. Por ejemplo, en futuras versiones podría agregarse una base de datos interna o un motor de análisis forense avanzado sin modificar la estructura de la GUI.

La decisión de separar las lógicas del programa se enfoca en evitar cuelgues y mantener un

código limpio, de forma que sea fácilmente legible para cualquier desarrollador que quiera aportar al proyecto, esto además de que el usar hilos de procesamiento asegura que el usuario pueda seguir usando la herramienta mientras se realizan distintos procesos, siendo esto principalmente utilizado para la implementación de un modelo tabular con actualización progresiva para mejorar el rendimiento, además de que esto ayuda a simplificar el testing independiente de cada módulo, reduciendo el acoplamiento del código.

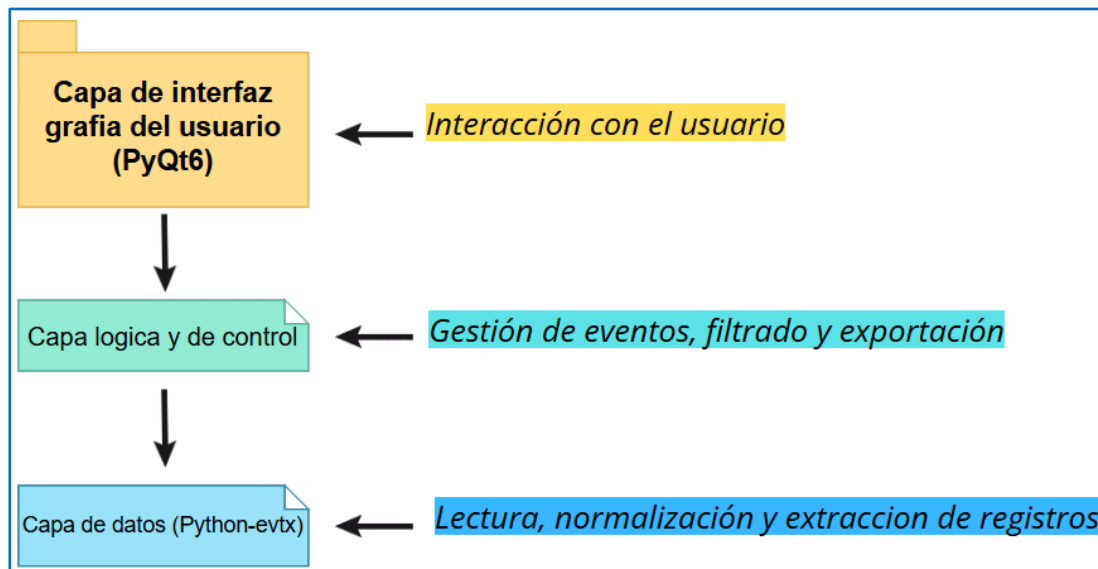


Gráfico 1. Visión general de la arquitectura

ii. Componentes principales

El programa se compone principalmente por 3 módulos, dichos módulos contienen diferentes funciones, todas cumplen una función enfocada en las lógicas de las capas detalladas anteriormente, dichos componentes son:

	Funcion principal	Descripcion
main.py	Núcleo de inicialización y permisos	Este módulo actúa como punto de entrada del programa. Su función principal es verificar si la aplicación se está ejecutando con privilegios de administrador, necesarios para acceder a los registros de Windows como System o Security. Si el programa no cuenta con dichos permisos, se relanza automáticamente utilizando ShellExecuteW y el intérprete pythonw.exe, lo que permite elevar privilegios sin mantener una consola visible. Este proceso evita que el usuario deba ejecutar manualmente la aplicación como administrador, mejorando la experiencia de uso.
main_window.py	Contenido	Contiene la clase principal MainWindow, que define toda la estructura de la GUI. Incluye métodos específicos para convertir los eventos leídos en estructuras manipulables (listas de diccionarios), así como para actualizar dinámicamente las cabeceras de la tabla según los datos detectados.
	Interfaz gráfica y control de flujo	Un menú principal con opciones de carga de registros desde el sistema o desde un archivo .evtx. Un botón de filtrado para buscar coincidencias específicas dentro de los registros cargados. Un botón de exportación, que permite guardar los resultados visibles en formato CSV. Una tabla de eventos, implementada con QTableWidgetItem, que presenta de manera estructurada los registros extraídos. Una ventana de texto inferior para mostrar el XML completo del evento seleccionado.
event_log_reader.py	Acceso directo a los registros del sistema	Este módulo implementa la clase EventLogReader, que se comunica directamente con las API nativas de Windows mediante pywin32. El método read_channel() es el corazón de este módulo. Determina si el parámetro recibido corresponde a un canal activo del sistema o a un archivo .evtx almacenado, adaptando la lectura a cada caso. Los eventos se obtienen a través de win32evtlog.OpenEventLog() y se recorren con ReadEventLog(), utilizando las banderas EVENTLOG_BACKWARDS_READ y EVENTLOG_SEQUENTIAL_READ. Cada evento se convierte en un objeto EventRecord, que contiene campos como la fuente, la fecha, el ID del evento y su descripción textual.

Tabla 1. Funcionalidad de módulos

d. Metodología de trabajo.

El proyecto se desarrolló bajo un enfoque iterativo e incremental, inspirado en metodologías ágiles. Se plantearon entregas parciales que permitieron validar progresivamente cada funcionalidad.

Fase	Descripción
Planificación y análisis	Se definieron los objetivos y se estudiaron los requerimientos técnicos necesarios para la lectura y manipulación de archivos EVTX. Se analizaron librerías existentes y se evaluó su compatibilidad con Windows.
Diseño	Se estableció la estructura modular del sistema y se elaboraron diagramas de flujo y prototipos de interfaz. En esta etapa se priorizó la claridad del diseño y la facilidad de uso para los analistas forenses.
Implementación	Se programaron los módulos en orden de dependencia, comenzando por <code>event_log_reader.py</code> y finalizando con <code>main.py</code> . Cada módulo fue probado de manera aislada antes de integrarlo al sistema principal.
Integración y validación	Se unieron los módulos y se realizaron pruebas funcionales completas, comprobando la correcta interacción entre la interfaz y la capa de acceso a datos.
Documentación y optimización	Finalmente, se documentaron los métodos, se optimizaron los tiempos de lectura y se mejoró el manejo de errores para garantizar una experiencia fluida y profesional.

Tabla 2. Fases de desarrollo

e. Gestión de errores y pruebas.

El sistema fue diseñado para manejar errores de forma segura y controlada, garantizando que los fallos no provoquen interrupciones ni pérdida de información. Las funcionalidades que se encargan de la gestión de errores más relevantes son:

- **Gestión de permisos:** el programa se relanza automáticamente con privilegios administrativos si detecta restricciones de acceso, evitando errores de lectura en canales protegidos, tal es el caso del canal Security.

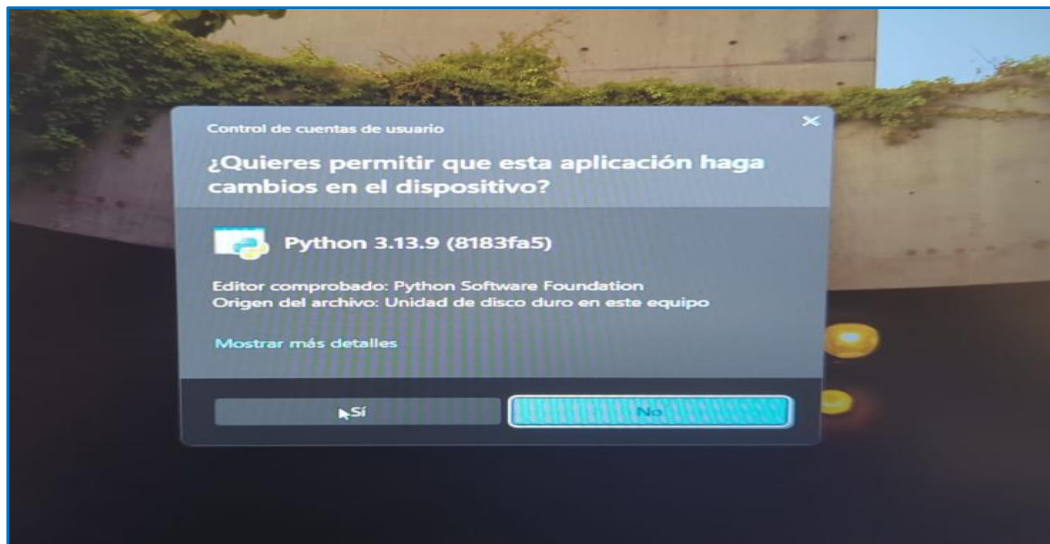


Gráfico 2. Verificación de permisos de ejecución

- Manejo de archivos bloqueados: antes de leer un archivo EVTX, se crea una copia temporal con `tempfile.gettempdir()` para evitar conflictos con el sistema operativo.

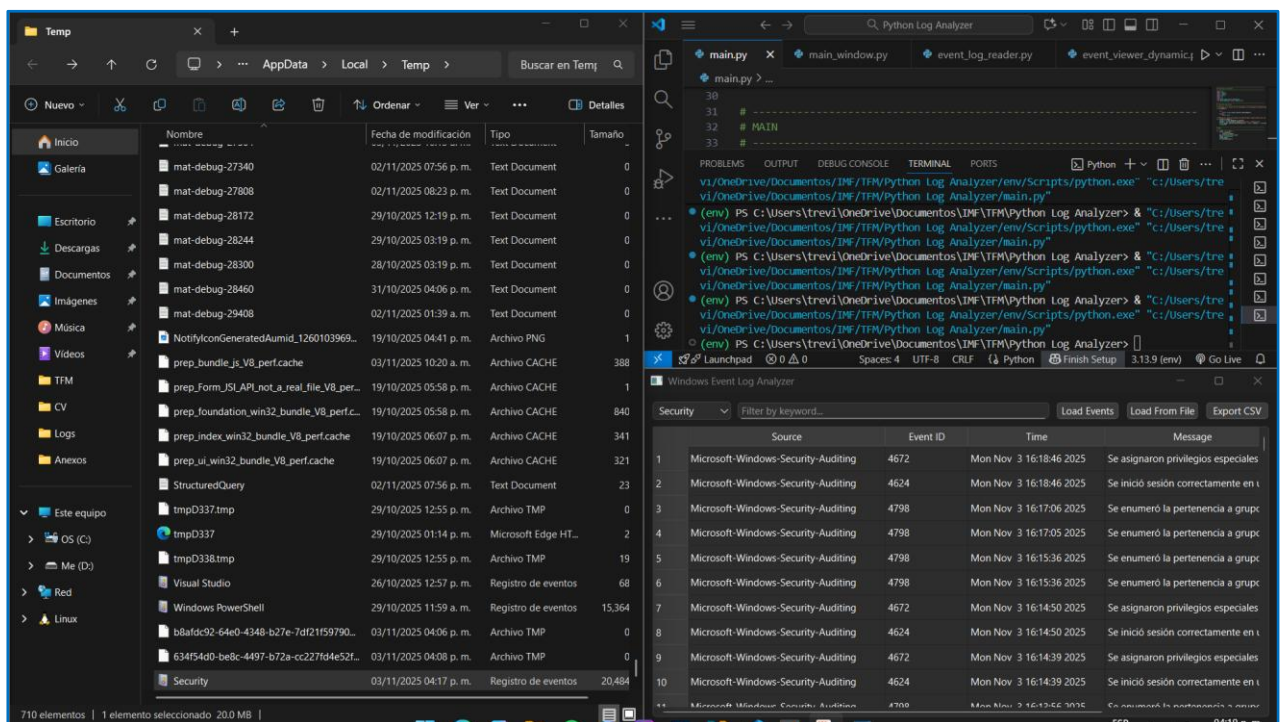


Gráfico 3. Manejo de archivos temporales

- Validación de dependencias: si alguna librería esencial (por ejemplo, python-evtx) no está instalada, se notifica al usuario mediante una ventana emergente con instrucciones para resolver el problema.

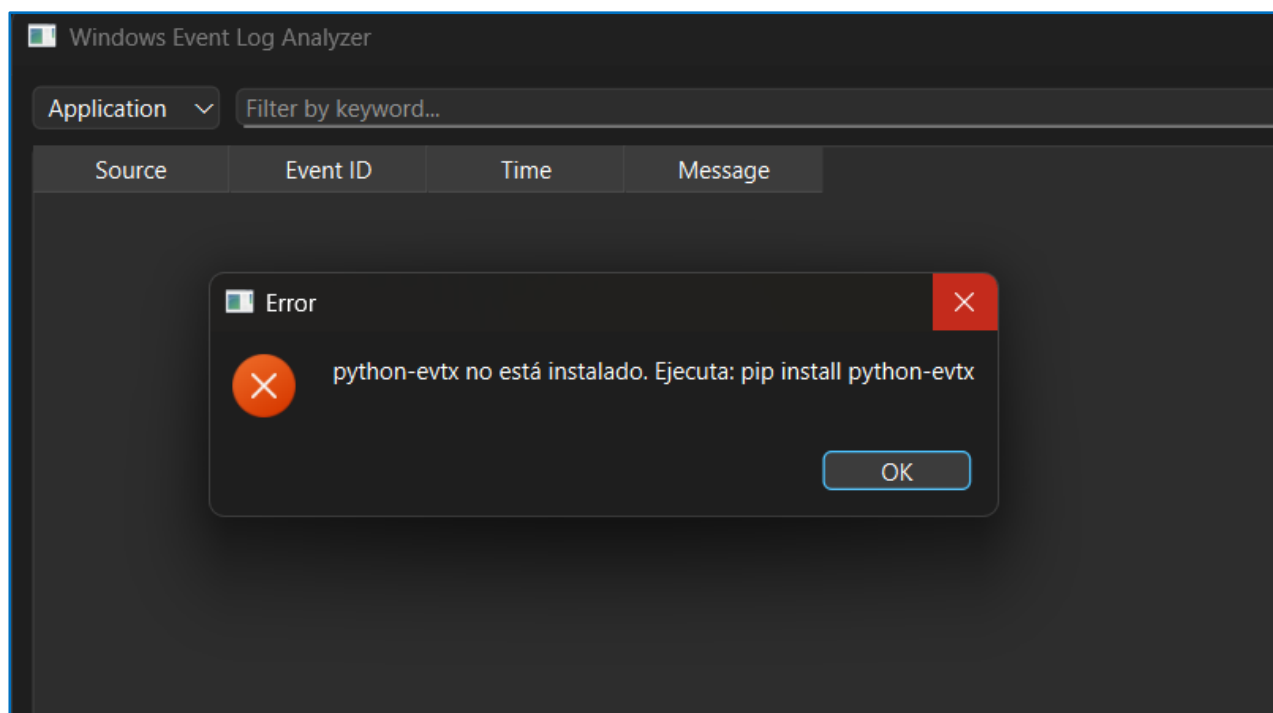


Gráfico 4. Validación de dependencias

- Control de excepciones: cada proceso de lectura y normalización está envuelto en bloques try/except, garantizando que los errores sean informados sin interrumpir la ejecución.

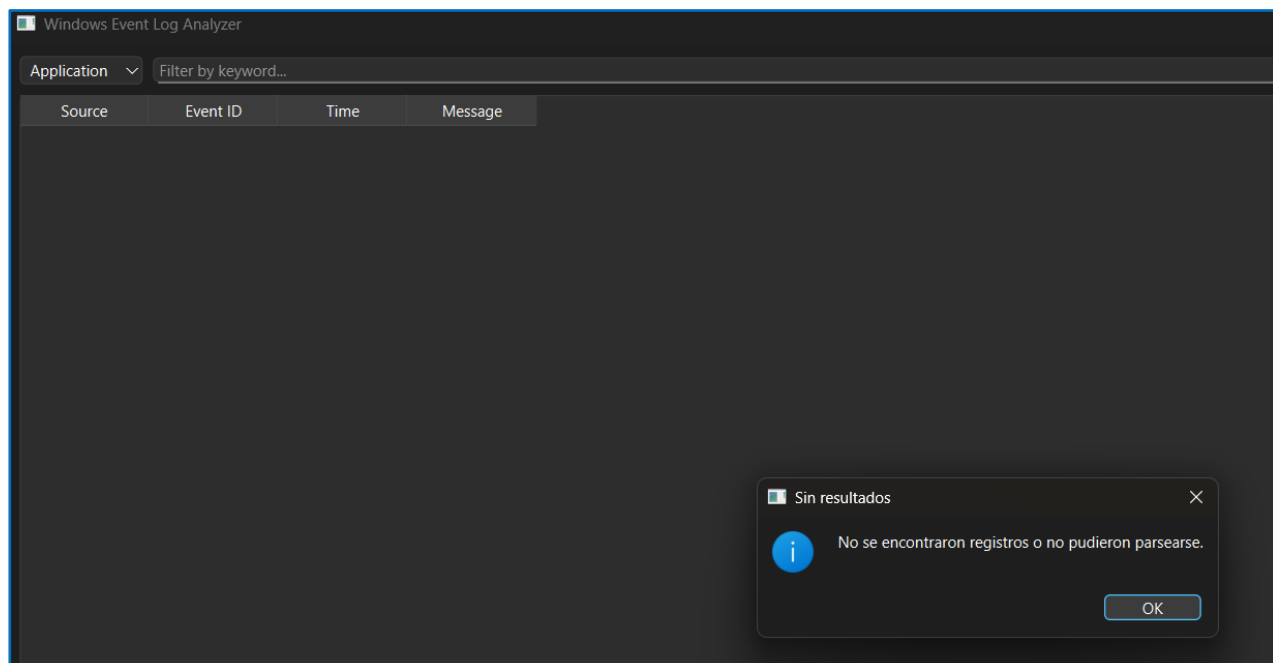


Gráfico 5. Ejemplo de manejo de errores

Al momento de realizar las pruebas del programa desarrollado, destacan las siguientes pruebas de rendimiento y funcionamiento:

- Funcionales: Se verifica la lectura de eventos desde los 3 canales principales, así

como desde la carga/apertura de archivos de eventos de sistema (EVTX) externos al propio sistema.

- Rendimiento: Se compararon los tiempos de ejecución/carga de archivos de distintos tamaños, siendo desde 10MB a 100MB con un promedio de 10 a 30 segundos.
- Comparación: Se contrastan los resultados con el visor de eventos de Windows, mediante dicha comparación, se confirma que los registros coinciden en contenido y formato.
- Robustez: Se utilizan archivos corruptos o con permisos limitados para verificar la estabilidad del sistema, cabe mencionar que, para indicar el funcionamiento del programa durante la lectura de archivos robustos, se cuenta con componentes visuales, tal es el caso de barras de carga.

La combinación de los materiales utilizados y la metodología aplicada permitió construir una herramienta sólida, eficiente y útil para el análisis de eventos de Windows. Cada elección tecnológica estuvo respaldada por una razón práctica, y los métodos empleados garantizan que el software no solo funcione, sino que puede evolucionar sin comprometer su estabilidad.

6. Resultados obtenidos.

La construcción de la herramienta permitió obtener un conjunto de resultados tanto técnicos como funcionales que demuestran la validez de la propuesta inicial y la utilidad práctica del proyecto. Durante el proceso de diseño, implementación y validación, fue posible comprobar que la herramienta cumple con los objetivos planteados, que ofrece una experiencia de uso fluida y que puede servir como apoyo real en escenarios de análisis forense o de auditoría de seguridad. En esta sección se describen detalladamente todos los resultados obtenidos, incluyendo el comportamiento de la interfaz, la funcionalidad del sistema, la eficiencia en la carga de eventos, la respuesta del programa ante archivos grandes y los aspectos que requieren mejora o representan oportunidades para futuros desarrollos.

a. Descripción de herramienta.

La herramienta desarrollada es una aplicación gráfica que permite visualizar de manera clara y estructurada los eventos generados por sistemas Windows, tanto desde archivos EVTX como desde los canales de eventos del propio sistema. Está construida con Python y PyQt6, lo que permite una integración sólida con componentes visuales modernos, una interfaz limpia, y un funcionamiento estable incluso cuando se manejan grandes volúmenes de información.

La idea principal fue construir una aplicación que permitiera navegar los logs de una manera más humana y ordenada, sin depender de herramientas pesadas o poco intuitivas. Para lograrlo, diseñé una interfaz dividida en dos paneles principales: una tabla de eventos y un visor XML jerárquico. Esto permite que el usuario pase de una vista general a una vista detallada con solo seleccionar un evento.

Diseño general de la interfaz:

La interfaz final está dividida en tres áreas principales:

1. Una barra superior de controles, desde la cual el usuario puede:
 - Cargar archivos EVTX,
 - Leer canales del sistema,
 - Aplicar filtros,
 - Realizar búsquedas globales,
 - Exportar resultados
2. Una vista de tabla, que presenta los eventos en formato tabular, permitiendo ordenar por columnas, aplicar filtros individuales, resaltar coincidencias y navegar rápidamente entre

miles de registros.

3. Una vista XML tipo árbol, donde se muestra la estructura completa del evento seleccionado, permitiendo al usuario explorar parámetros internos más complejos, campos ocultos y atributos específicos de cada registro, siendo esto una manera más moderna de visualizar la información de los eventos, esto en lugar de la información en formato crudo de XML.

Este diseño permite un flujo de trabajo natural: cargar eventos → filtrarlos o buscar algo específico → explorar el XML → extraer conclusiones. No requiere conocimientos previos avanzados, y cualquier persona habituada a herramientas gráficas puede entenderla rápidamente.

Las características principales de la herramienta ubicadas en cada uno de los componentes gráficos de la herramienta son:

- **Carga de archivos EVTX externos** y lectura directa de canales del sistema (Security, System, Application).
- **Representación estructurada de los eventos** en una tabla dinámica con columnas adaptadas automáticamente.
- **Visor XML interactivo**, donde el usuario puede explorar nodos, atributos y valores tal como aparecen en el registro original.
- **Búsqueda global** con posibilidad de filtrar dentro de los campos visibles o dentro del XML crudo.
- **Sistema de filtros avanzados**, útiles para análisis forense (mayor que, menor que, contiene, entre fechas, etc.).
- **Resaltado automático de eventos críticos**, como IDs relacionados con autenticación (4624, 4625), creación de procesos (4688) o borrado del log (1102).
- **Exportación de resultados filtrados a CSV** para documentar hallazgos o trasladarlos a otras herramientas.
- **Interfaz responsiva y estética**, basada en un archivo style.qss que le da un aspecto moderno y profesional.

En general, la herramienta en su estado actual funciona como un visor de logs ágil y práctico, que responde bien a necesidades reales dentro del análisis forense y de una manera que no requiera una gran experiencia del usuario.

i. Anexos de interfaz.

La interfaz final presenta un diseño moderno, oscuro y limpio, optimizado para largas sesiones de análisis. Este diseño no es solo estético: el modo oscuro reduce la fatiga visual, especialmente al trabajar con grandes cantidades de texto o valores numéricos.

En las pruebas se observó que:

- El usuario puede identificar fácilmente qué elementos son interactivos;
- La tabla ofrece un contraste claro entre filas;
- La vista XML mantiene indentación coherente;
- Los botones tienen un tamaño adecuado para evitar clics erróneos;
- Los estilos visuales permanecen uniformes entre las diferentes áreas de la aplicación.

Además, los iconos y recursos gráficos son ligeros, lo que evita ralentizaciones innecesarias. La interfaz fue validada en pantallas HD y FHD, manteniendo buena distribución y legibilidad.

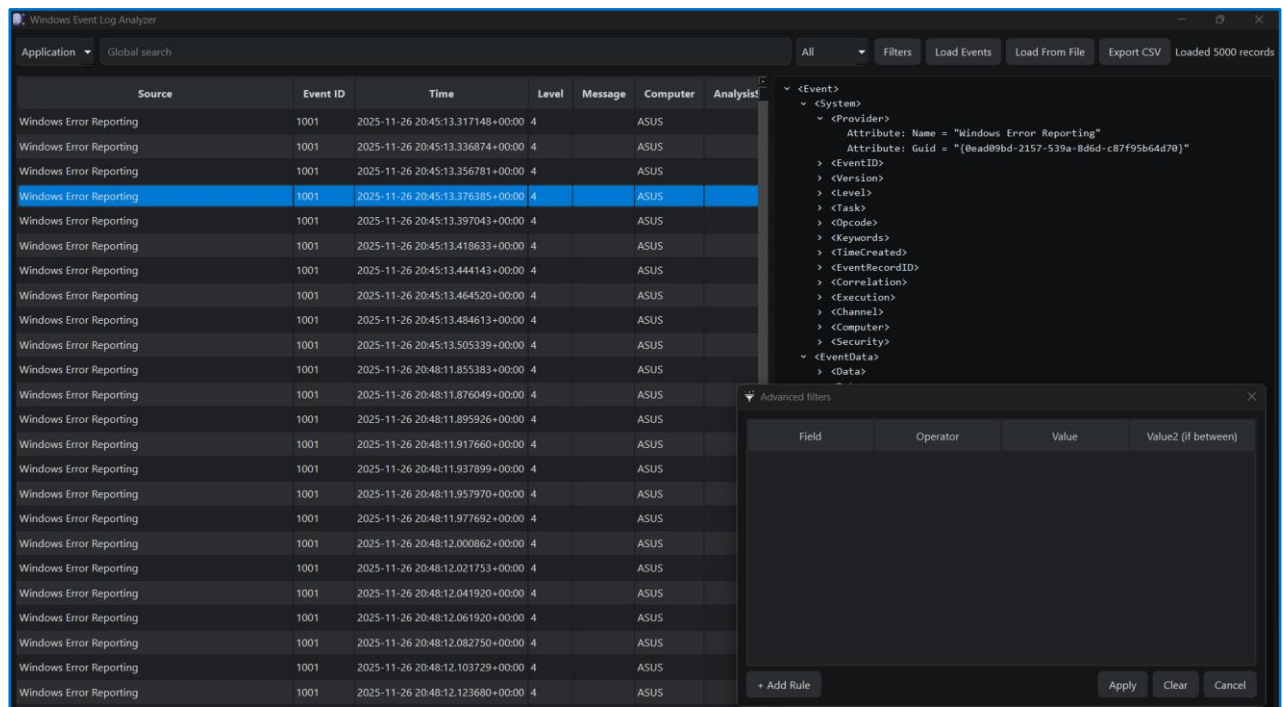


Gráfico 6. Propuesta de interfaz grafica

De manera rápida, se pueden definir los componentes visuales de la imagen anexada con los componentes descritos en el diseño general de la interfaz como:

- Pantalla principal de carga: Se muestra la ventana inicial con la barra superior de controles, donde se encuentran las opciones para cargar canales o archivos, realizar búsquedas, aplicar filtros avanzados y exportar datos.
- Tabla de eventos: Vista con los registros ya cargados, organizados por columnas como *EventID*, *Source*, *TimeCreated*, *Level*, entre otras. Esta tabla está diseñada para manejar

desde cientos hasta decenas de miles de eventos.

- Visor XML en árbol: Cuando el usuario selecciona un evento, a la derecha aparece el XML estructurado con nodos expandibles, permitiendo examinar cada campo del evento con claridad.
- Ventana de filtros avanzados: Ejemplo del cuadro de diálogo que permite construir reglas de filtrado complejas, útiles para investigaciones más profundas.
- Búsqueda global: Se muestra cómo la herramienta resalta coincidencias dentro del XML o dentro de la tabla, según lo que seleccione el usuario.

b. Evaluación funcional.

La evaluación funcional de la herramienta se hizo probando diferentes escenarios de uso comunes en el análisis de eventos:

1. Carga de archivos grandes

Se probaron archivos EVTX de más de 20.000 eventos, por fines demostrativos, decidí limitar la lectura de eventos a 5000 eventos por canal o archivo. La herramienta logró:

- Procesarlos sin bloquear la interfaz (gracias al uso de hilos)
- Generar las columnas dinámicamente
- Realizar búsquedas en tiempos razonables

Aunque no está pensada para manejar millones de eventos, funciona bien dentro del rango habitual de análisis forense.

2. Lectura de eventos del canal Security

Los eventos más importantes para análisis (4624, 4625, 4688, 4672, etc.) se cargan correctamente y aparecen con destaque visual. Se verificó que los tiempos de carga son adecuados y que el árbol XML se genera sin errores, esto al concordar con los datos en formato crudo.

3. Funcionamiento de las búsquedas

La búsqueda global es rápida incluso con miles de registros. Se comprobó:

- Búsqueda por texto
- Búsqueda por campo
- Coincidencias dentro del XML

El resaltado de texto dentro del XML ayuda bastante a navegar los resultados.

4. Filtros avanzados

Se probó el filtrado por:

- Rango de fechas
- Comparación de números
- Coincidencias parciales

Los filtros funcionan correctamente, y permiten llegar a conclusiones sin necesidad de exportar los datos.

5. Exportación a CSV

Los datos filtrados se exportan sin perder la información visible.

El CSV resultante preserva:

- Filas filtradas
- Encabezados
- Orden actual
- Codificación adecuada para Excel

La herramienta cumple su propósito, esto debido a que permite analizar registros de eventos de manera rápida, clara y organizada, incluso para alguien que no esté familiarizado con el formato EVTX o con el visor de eventos de Windows.

c. Áreas de mejora.

Aunque la herramienta funciona bien, durante las pruebas surgieron varias oportunidades claras de mejora:

6. Mejorar el rendimiento en logs extremadamente grandes

Archivos con cientos de miles de eventos pueden ralentizar la interfaz.

Posibles soluciones:

- Paginación de resultados,
- Carga diferida (“lazy load”),
- Indexación previa de eventos.

7. Implementar un sistema de correlación

Sería útil que la herramienta pueda relacionar eventos similares o dependientes automáticamente, por ejemplo:

- Correlación entre 4624 → 4672 → 4688
- O correlación mediante LogonID.

Esto ahorraría tiempo al analista.

8. Añadir búsqueda semántica

Actualmente la búsqueda es textual.

Una futura mejora sería:

- Buscar por tipos de eventos,
- Buscar "intentos de autenticación fallidos",
- O identificar automáticamente patrones de ataque conocidos.

9. Soporte para otros logs además de EVTX

La herramienta podría extenderse para soportar:

- Sysmon
- Registros de PowerShell
- Archivos .csv de auditorías personalizadas

10. Generación automática de reportes

Una función que exporte:

- Estadísticas,
- Eventos sospechosos,
- Líneas de tiempo,
- Conclusiones preliminares

11. Mejor manejo de hilos

Aunque la app es estable, algunos procesos todavía se podrían paralelizar para mejorar la fluidez general.

12. Firma digital del ejecutable

Para distribuir la herramienta en entornos corporativos es conveniente firmarla con un certificado digital.

En conjunto, los resultados obtenidos demuestran que la herramienta es estable, funcional y útil en escenarios reales. Permite analizar eventos de Windows de manera más fluida que el visor nativo, facilita la interpretación del XML, y ofrece capacidades avanzadas que son especialmente valiosas para estudiantes, analistas de seguridad y profesionales que requieren una herramienta ligera pero potente.

7. Modelos utilizados.

En esta parte explico los modelos que utilicé para diseñar y construir la herramienta. Más allá del código, me interesaba dejar claro qué hay detrás: cómo se conceptualiza el sistema, cómo se estructura internamente la información, cómo se organiza la interfaz para que sea fácil de usar y, finalmente, cómo fluye todo dentro del programa una vez que se ejecuta. Esta sección sirve para entender que no se trata solo de “leer logs”, sino de un conjunto de decisiones técnicas pensadas para facilitar el análisis forense.

a. Modelo conceptual.

Al empezar este proyecto, lo primero que tuve que hacer fue visualizar la herramienta desde lo más abstracto: ¿qué problema quiero resolver y cómo quiero que el programa procese la información?

De forma conceptual, la herramienta funciona como un puente entre los eventos del sistema operativo Windows y el analista que necesita interpretarlos. Los registros en formato EVTX están llenos de detalles, pero son complicados de leer directamente (incluso usando el lector de eventos nativo de Windows), por eso el objetivo era transformarlos en algo más accesible, ordenado y útil.

Para la planeación y desarrollo de la solución propuesta me enfoque en la fragmentación de la tarea completa, es decir, me enfoque en cuatro módulos principales que derivaron en distintas y más pequeñas funcionalidades del programa

Componente	Descripción	Derivados
Origen de los datos.	Define la fuente de los datos que el programa procesara.	Identificación de los canales principales de información. Definición de archivos aceptados
Procesamiento de la información.	Una vez cumplida la carga de información, ya sea mediante archivos o canal, el programa traduce la información XML.	Normalizar campos clave como EventID, TimeCreated o Level. Identificar los datos dentro de System y EventData. Extraer mensajes cuando estén disponibles. Mantener el XML original intacto.
Interfaz gráfica.	Una vez procesados, los datos llegan a la interfaz, donde el usuario puede explorar la información sin necesidad de ir evento por evento en formato crudo.	La tabla de eventos El árbol XML Filtros y búsquedas Controles de carga y exportación
Presentación y salida.	El programa presenta los eventos de manera clara y permite exportarlos	

Tabla 3. Modelo de conceptual

b. Modelo de datos.

Los eventos del sistema operativo Windows son extremadamente variados. Un Event ID 4624 no tiene los mismos campos que un 4688, y tampoco los mismos que un error del sistema o un evento de aplicación. Por eso necesitaba un modelo de datos que fuera **flexible**, que se adaptara y que no perdiera información. Esto plantea un problema al momento de normalizar todos los eventos a un formato compartido, por lo que decidí que la estructura más natural para esto fuera un diccionario de Python, lo que permite que cada evento se pueda representar de una manera similar

```
{
    "Source": "Microsoft-Windows-Security-Auditing",
    "EventID": "4624",
    "TimeCreated": "2023-10-11T12:22:33.000Z",
    "Level": "Information",
    "Message": "An account was successfully logged on.",
    "__raw_xml": "<Event>...</Event>",
    "TargetUserName": "Pedro",
    ...
}
```

Al definir los eventos de esta manera, puedo almacenar y procesar solo los campos que, si existen y en caso de que haya algún otro relevante, agregarlo automáticamente, también es importante resaltar que este diccionario permite mantener el evento original sin modificar (para la vista en crudo del XML al hacer doble clic sobre cualquier evento) y así poder reconstruir rápidamente la tabla con todos los registros.

La definición de este modelo implica que no todos los eventos tienen los mismos atributos, por lo cual, el programa está diseñado para analizar todos los eventos cargados, detectando claves que aparecen y construyendo dinámicamente la tabla principal con las columnas necesarias sin dejar de priorizar los campos esenciales como el ID, el tiempo y mensaje, de esta manera se evitan errores de lectura y de llenado de tablas al haber eventos que no coinciden en atributos.

c. Modelo de interfaz.

La interfaz gráfica es el punto donde el usuario realmente interactúa con la herramienta, así que me enfoqué en que fuera clara, intuitiva y que permitiera hacer análisis rápido, por lo cual, decidí dividirla en 3 grandes partes (con 2 derivados pequeños)

i. Barra superior

Para el desarrollo de esta sección, decidí enfocarme en componentes con funciones más usadas, de forma que estuvieran arriba y fueran siempre visibles:

- Selección de canal a leer en el sistema local.
- Búsqueda global basada simplemente en texto.
- Botón de filtros avanzados (Abre ventana para agregar filtros a el archivo abierto).
- Botón para cargar datos del canal seleccionado.
- Botón para seleccionar archivos que no sean alguno de los 3 canales principales.
- Botón de exportación a CSV.

ii. Panel izquierdo de tabla de eventos

En esta sección me enfoqué principalmente en mostrar una tabla simple en donde se ve la información normalizada:

- Generación de columnas dinámicas.
- Filtros por encabezado (clic derecho).
- Resaltado de eventos críticos.
- Ajuste de ancho de tabla automático.
- Despliegue de ventana con el evento crudo (doble clic).

iii. Panel izquierdo para árbol XML

A pesar de que existe la opción de ver la información XML de cada evento mediante el doble clic en dicho evento, para una representación más ágil de leer también decidí incluir un visor de eventos mediante un árbol XML, facilitando la lectura de los bloques de cada evento:

- Resaltado de coincidencias.
- Expansión manual y automática del ancho de la sección.
- Eliminación de namespaces innecesarios.

iv. Mockups y productos mínimos viables.

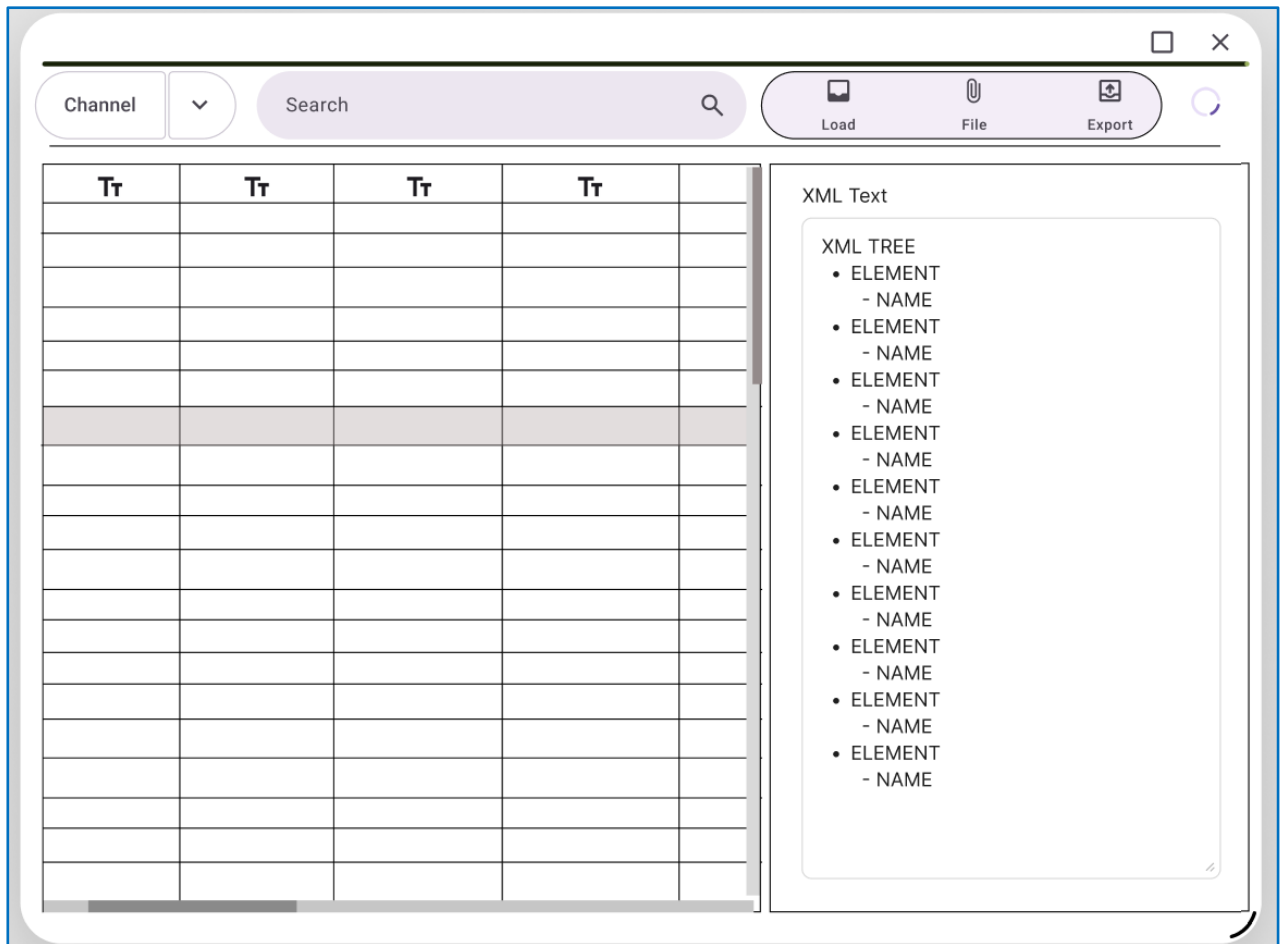


Gráfico 7. Mockup del programa

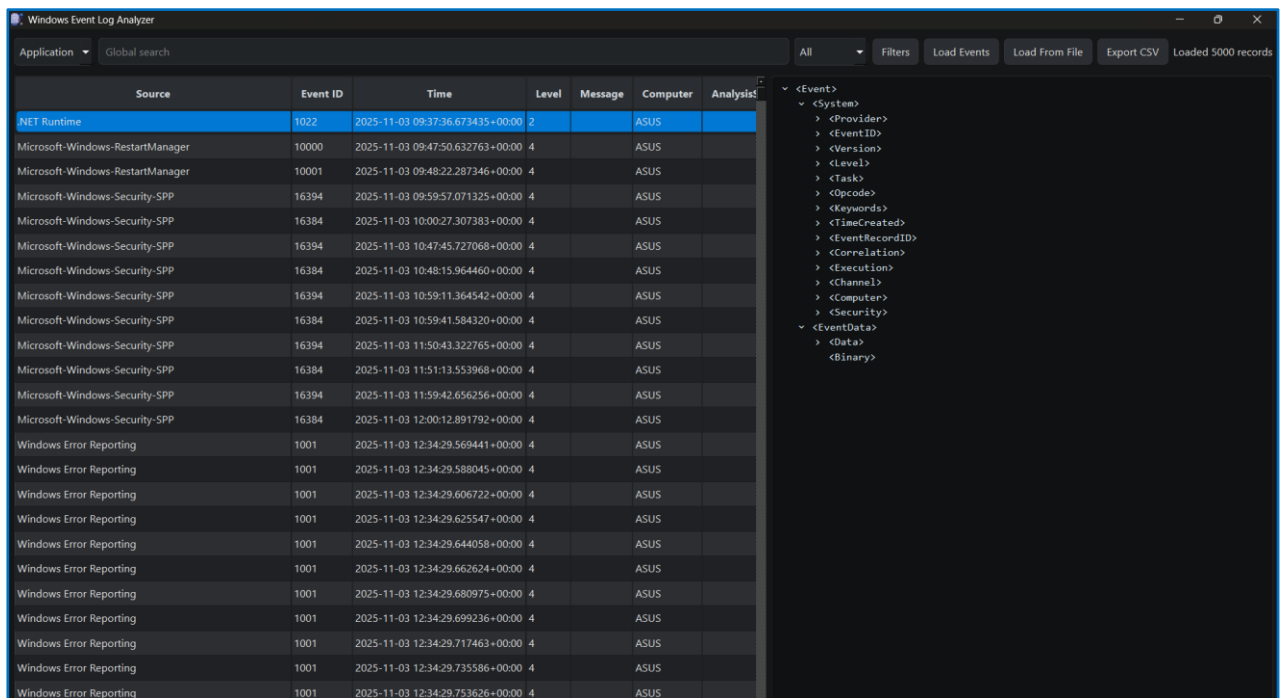


Gráfico 8. Interfaz propuesta

d. Modelo lógico de funcionamiento.

Para explicar cómo funciona el programa internamente, lo describo como una serie de pasos que ocurren cada vez que el usuario carga un archivo o un canal:

Paso	Descripcion
El usuario selecciona la fuente	El usuario puede seleccionar un canal del sistema o un archivo externo.
Se crea una copia temporal del archivo	Esto se hace para evitar problemas si el archivo está en uso, además respeta buenas prácticas forenses: no se modifica el archivo original.
Se lee el log	Dependiendo de la disponibilidad de módulos instalados en el sistema, se utiliza python-evtx o win32evtlog para la lectura.
Procesamiento de eventos XML	Se normalizan los campos.
	Se extraen los valores de System y EventData.
	Se obtiene el mensaje cuando existe.
	Se agrega el XML original al registro.
Generación dinámica de columnas	El programa examina todos los eventos y determina qué columnas serán visibles en la tabla.
Aplicación de filtros	Controles por encabezado.
	Filtros avanzados (mayor que, menor que, entre, etc).
	Búsqueda global en tabla, XML o ambos.
Resaltado de eventos	EventID's como 4625, 4688 o 1102 aparecen con colores especiales, porque suelen ser indicadores de amenaza.
Actualización de interfaz	Finalmente el usuario puede ver las 3 secciones principales con agregados menores como el estado actual del programa y el número de registros.

Tabla 4. Procesamiento del programa.

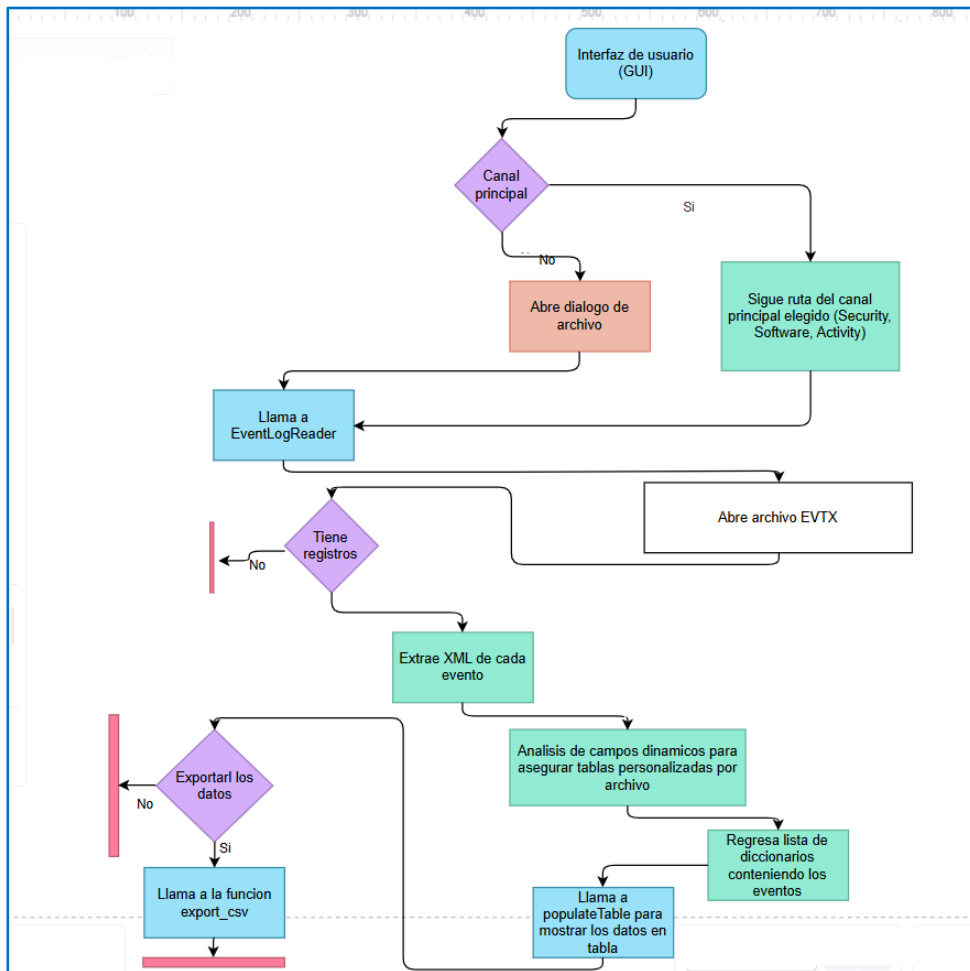


Gráfico 9. Flujo de datos.

8. Discusión y conclusiones.

La creación de esta herramienta permitió no solo desarrollar una aplicación funcional para el análisis de eventos de Windows, sino también reflexionar profundamente sobre el proceso completo: desde el planteamiento del problema hasta la validación de resultados. En esta sección se discuten los aprendizajes obtenidos, las interpretaciones derivadas de los resultados funcionales y la relevancia práctica del proyecto en el contexto de la informática forense, la administración de sistemas y la ciberseguridad.

a. Interpretación de resultados.

Los resultados obtenidos demuestran con claridad que la herramienta cumple con los objetivos planteados y que representa una alternativa viable, ligera y eficiente al visor nativo de Windows (Event Viewer). A partir de las pruebas realizadas, fue posible identificar varios puntos clave que permiten interpretar el alcance real del proyecto.

Al momento de reflexionar sobre lo conseguido con esta herramienta, no puedo evitar pensar en los objetivos planteados, esto ya que, a final de cuentas fueron los que motivaron la idea y desarrollo del producto obtenido, cada uno de los objetivos contribuyo a la agregación de una característica que fue integrada o desarrollada para que el usuario final pueda hacer uso de ella.

- **Facilidad de uso y accesibilidad**

Una de las primeras conclusiones que pueden extraerse es que la interfaz desarrollada logra simplificar significativamente el proceso de análisis de eventos. Al organizar toda la información en una tabla clara y combinarla con un visor XML ordenado, se redujo drásticamente la carga cognitiva necesaria para revisar registros complejos. Esto es especialmente relevante si se considera que muchas herramientas profesionales presentan interfaces densas, llenas de opciones o con funciones distribuidas de manera poco intuitiva.

En las pruebas se observó es fácil seguir el flujo de acciones para:

- Cargar eventos
- Aplicar filtros
- Navegar en el XML
- Interpretar la información sin necesidad de consultar documentación externa.

Esto indica que la herramienta no solo funciona bien, sino que también está bien diseñada desde el punto de vista de la experiencia de usuario, tomando en cuenta desde la conceptualización de la herramienta esto como un punto clave para su desarrollo.

- **Rendimiento técnico y estabilidad**

Otra interpretación relevante es la capacidad de la herramienta para manejar volúmenes significativos de datos. Aunque no se buscaba competir con plataformas empresariales, la aplicación fue capaz de cargar y procesar miles de eventos sin bloqueos o caídas inesperadas, lo cual es un indicador de que la arquitectura modular y el uso de hilos independientes fueron decisiones acertadas, no obstante, para fines demostrativos, decidí limitar la lectura de los archivos a 5000 eventos por canal o fichero de eventos, esto.

La estabilidad frente a errores como logs dañados, archivos incompletos o formatos atípicos demuestra que el sistema implementado de manejo de excepciones cumple su función. El hecho de que estas situaciones no rompan la aplicación indica una madurez importante del proyecto y un entendimiento claro de las necesidades de una herramienta orientada al análisis forense.

- **Valor práctico en un contexto real de seguridad**

Quizá uno de los objetivos más relevantes es el valor práctico que esta herramienta puede tener dentro de los escenarios reales en los que se necesita de una herramienta que pueda brindar un análisis rápido y eficaz de eventos de seguridad en caso de que no se cuente con tiempo o experiencia suficiente en una de las herramientas que ofrecen una funcionalidad similar, pero con una curva de aprendizaje mayor para el manejo de esta. Durante las pruebas se identificó que el sistema:

- Facilita el análisis rápido de eventos críticos,
- Ayuda a interpretar comportamientos sospechosos mediante el resaltado visual y la vista del XML.

Eventos como inicios de sesión fallidos repetidos, ejecuciones de PowerShell inesperadas o la creación de procesos anómalos pueden identificarse con mucho menos esfuerzo gracias a este sistema. Esto indica que el visor no solo es útil para estudiantes, sino también para profesionales que necesiten un análisis preliminar inmediato en un entorno donde el tiempo es fundamental.

- **Fortalezas detectadas a lo largo del proyecto**

El análisis del desarrollo revela varias fortalezas notables que al principio no tome tanto en cuenta, pero a lo largo del proyecto fueron tomando mayor protagonismo en cuanto a las características que posee la herramienta como puntos de diferenciación a la hora de elegir utilizarla:

- Modularidad del código: facilita la lectura y modificación del sistema sin afectar lo ya

construido, incluso reutilizando componentes ya existentes a los que se podrían agregar.

- Interfaz intuitiva: mejora la experiencia del usuario y reduce el tiempo de aprendizaje en comparación a otras herramientas que cumplen con un propósito similar al planteado.
- Potencia en el filtrado y búsqueda: brinda capacidades superiores al visor nativo para la búsqueda de eventos, ya sea mediante texto o mediante los filtros avanzados que fueron implementados para facilitar al usuario su experiencia con la herramienta.
- Manejo robusto del XML: permite un análisis profundo de cada evento, esto mediante la vista del árbol XML ubicado a la derecha de la interfaz al momento de hacer clic en un evento, sin embargo, también como una funcionalidad no tan mencionada, al momento de hacer doble clic a un evento en la tabla de eventos, se abrirá una ventana que muestra el texto crudo del evento en formato XML, esto es un extra a lo planteado originalmente, pero darle al usuario este tipo de funcionalidades puede ayudar al momento de estar más avanzado en la curva de aprendizaje, ya sea de interpretación de datos o uso de la herramienta.
- Transparencia visual: el resaltado de eventos críticos mejora la investigación e interpretación de los datos, esto aunado al filtrado de los eventos mediante los distintos campos que poseen.

Estas fortalezas indican que la solución no solo funciona, sino que está bien fundamentada tanto técnica como conceptualmente, permitiendo mejorar la herramienta tomando en cuenta aspectos detallados aquí.

- **Limitaciones interpretadas durante las pruebas**

Como cualquier proyecto técnico, se identificaron también ciertas limitaciones que ayudan a comprender el alcance de la herramienta, de forma que se pueden definir las áreas en las que se puede mejorar la herramienta en futuras actualizaciones que brinden estas limitaciones a puntos donde dejen de mostrarse inconvenientes. Por ejemplo:

- Aunque la carga de archivos medianos es rápida, los muy grandes pueden tardar más de lo deseado, esta es otra de las razones para limitar la lectura actual a 5000 eventos, sin embargo, esto no significa que la carga sea lenta, sino que no está totalmente optimizada para archivos demasiado grandes o que contengan millones de eventos, los cuales pueden aparecer en casos reales.
- La herramienta se centra en análisis local, no en escenarios distribuidos o remotos, dígame el hecho de que la herramienta, en el repositorio cuenta solamente con versión en ficheros

individuales de Python, de manera que no se cuenta con un ejecutable para distribución.

Estas limitaciones no afectan la validez del proyecto, pero sí permiten situarlo dentro del contexto adecuado: una herramienta ligera, de propósito analítico, no una plataforma empresarial de monitoreo que aún tiene áreas de mejora para continuar con líneas de desarrollo futuras.

De manera general, se puede reconocer el estado de la herramienta como que los resultados muestran que:

- La carga de archivos EVTX funciona de manera estable, incluso con volúmenes grandes de eventos.
- La visualización en dos paneles (tabla + XML) facilita la transición entre una vista general y una vista detallada.
- Las búsquedas y filtros permiten explorar patrones sin necesidad de recurrir a otras herramientas o scripts externos.
- El comportamiento de la aplicación durante la lectura de logs, aun cuando se trata de miles de eventos, demuestra que la arquitectura diseñada es sólida.
- El resaltado automático de eventos críticos ayuda a que un analista, incluso sin mucha experiencia, identifique actividad sospechosa más rápidamente.

En general, los resultados apuntan a que la aplicación ofrece una experiencia de análisis práctica y eficiente, adecuada tanto para estudiantes como para analistas que necesitan una solución ligera para trabajar con logs de Windows.

b. Conclusiones generales.

Tras completar el desarrollo y evaluación del proyecto, es posible concluir que la herramienta cumple satisfactoriamente con los objetivos planteados y que representa un aporte real tanto para el aprendizaje como para la práctica profesional.

En primer lugar, se logró crear una herramienta accesible, potente en la medida de lo posible y estable que permite visualizar y analizar eventos de Windows con mayor fluidez que el visor nativo de una manera más amigable para el usuario. La integración entre la tabla y el visor XML facilita la interpretación de los registros y mejora significativamente la eficiencia operativa.

En segundo lugar, el proyecto demostró que herramientas basadas en Python y PyQt6 pueden ser perfectamente viables para análisis forense local, incluso cuando se manejan archivos de tamaño considerable. Esto abre el camino a que más estudiantes, analistas y profesionales puedan desarrollar sus propias soluciones a medida sin depender de plataformas costosas o complejas.

En tercer lugar, el proceso mismo de desarrollo permitió comprender en profundidad el

funcionamiento de los registros de eventos, su estructura interna, las dificultades que presentan y la importancia de contar con herramientas bien diseñadas para analizarlos. Esta comprensión se traduce en mejores prácticas, mayor eficiencia y una perspectiva más madura sobre el papel del análisis de logs en la ciberseguridad moderna.

Por último, este proyecto no solo validó su hipótesis inicial, sino que también dejó claro que existen amplias oportunidades para seguir expandiendo sus capacidades en el futuro. Aunque esta herramienta ya es completamente funcional, su arquitectura modular permite extenderla hacia soluciones más sofisticadas si se deseara y a mejorar las funcionalidades que, a pesar de cumplir con los objetivos planteados, están en área de mejora para futuras versiones o aportes.

En conjunto, los resultados y conclusiones evidencian que este proyecto representa un aporte académico y práctico, y que refleja la integración exitosa entre desarrollo de software, análisis forense y experiencia real en ciberseguridad en la rama de análisis forense.

Al finalizar el proyecto, puedo destacar varios puntos importantes:

- La herramienta es funcional, estable y cumple con los objetivos planteados, ofreciendo una forma clara de visualizar y analizar eventos de Windows sin depender de herramientas más complejas.
- El modelo de arquitectura basado en parseo XML + tabla dinámica + visor jerárquico funciona muy bien, y se adapta a cualquier tipo de evento.
- Las búsquedas avanzadas y el resaltado de eventos críticos mejoran significativamente el proceso de análisis, especialmente en situaciones donde se manejan archivos grandes.
- El proyecto demuestra que es posible crear una herramienta útil utilizando tecnologías accesibles como Python, PyQt6 y librerías de análisis de logs.
- Esta herramienta puede servir como base para proyectos más grandes, ya sea en investigación forense, ciberseguridad o enseñanza.

c. Líneas futuras de desarrollo.

Aunque la herramienta es totalmente funcional, quedó claro durante el proceso que hay varias áreas con gran potencial para ampliarse. Algunas ideas para versiones futuras incluyen:

1. Correlación automática de eventos

Detectar patrones como:

- Inicio de sesión → elevación de privilegios → ejecución de procesos
- Borrado de logs después de un evento sospechoso
- Relaciones entre LogonID y procesos

Esto sería un salto enorme hacia una herramienta más inteligente.

2. Integración con Sigma o MITRE ATT&CK

La herramienta podría marcar automáticamente eventos que coincidan con firmas Sigma o tácticas ATT&CK.

3. Generación de reportes automáticos

Exportar datos en otros formatos que permitan mostrar con más detalle:

- Eventos sospechosos
- Resúmenes por categoría
- Líneas de tiempo interactivas
- Estadísticas de logins, procesos y errores

4. Análisis de múltiples archivos simultáneamente

Poder cargar varios EVTX y correlacionarlos entre sí.

5. Optimización del rendimiento

Implementar mejoras de rendimiento en las tareas encargadas de:

- Paginación en la tabla,
- Indexación de eventos,
- Carga diferida del XML.

6. Implementación de un módulo Sysmon

Esto permitiría detectar actividad mucho más detallada, como conexiones de red, creación de procesos con hash, etc.

7. Firma digital e instalador profesional

Para permitir distribución segura en entornos empresariales.

En general puedo afirmar que el proyecto cumple con las funcionalidades principales con las que fue planeado, siendo una herramienta que ofrece un entorno sencillo para el análisis forense, además de ser un proyecto abierto a contribuciones para mejora continua, ya sea siguiendo las líneas de desarrollo futuras o alguna propuesta externa a cualquier involucrado directo del proyecto, en otras palabras, yo.

9. Referencias bibliográficas.

- Casey, E. (2011). *Digital Evidence and Computer Crime: Forensic Science, Computers and the Internet* (3rd ed.). Academic Press.
- Mandia, K., Prosise, C., & Pepe, M. (2014). *Incident Response and Computer Forensics* (3rd ed.). McGraw-Hill.
- Carvey, H. (2018). *Windows Forensic Analysis Toolkit* (4th ed.). Syngress.
- Harries, F. (2019). Event log analysis in cybersecurity: Understanding patterns of compromise. *Journal of Digital Forensics, Security and Law*, 14(1), 1–18.
- Institute for Applied Network Security. (2020). *Understanding Windows Event Logs for Security Monitoring*. IAINS.
- Microsoft. (2023). *Windows Event Logging architecture*. <https://learn.microsoft.com/en-us/windows/win32/eventlog/event-logging>
- Microsoft. (2023). *Event Viewer: Event ID reference*. <https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-ids>
- Russinovich, M., Solomon, D. A., & Ionescu, A. (2017). *Windows Internals, Part 1* (7th ed.). Microsoft Press.
- Russinovich, M., Solomon, D. A., & Ionescu, A. (2017). *Windows Internals, Part 2* (7th ed.). Microsoft Press.
- Microsoft. (2023). *Sysmon - System Monitor*. <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>
- Beale, J., & Foster, C. (2019). *The Elasticsearch 7.0 Reference Guide*. O'Reilly Media.
- Splunk. (2022). *Splunk Enterprise Security: Product documentation*. <https://docs.splunk.com/>
- Graylog. (2022). *Graylog documentation: Log management and SIEM*. <https://docs.graylog.org/>
- Hubbard, J., & Williams, B. (2020). Using Sysmon for advanced threat detection. *SANS Institute Whitepaper*.
- Summerfield, M. (2020). *Rapid GUI Programming with Python and Qt* (2nd ed.). Prentice Hall.
- Riverbank Computing. (2024). *PyQt6 documentation*. <https://www.riverbankcomputing.com/static/Docs/PyQt6/>
- Python Software Foundation. (2024). *Python 3.x documentation*. <https://docs.python.org/3/>
- Kramer, G. (2021). Best practices in multithreaded GUI applications using PyQt. *International Journal of Software Engineering*, 32(4), 55–69.
- Hurst, W. (2018). Understanding EVTX file structure for forensic analysis. *ACM Digital Forensics Conference Proceedings*, 22(3), 133–148.

Open Source Digital Forensics. (2022). *python-evtx: Library for parsing Windows XML EventLog files*. <https://github.com/williballenthin/python-evtx>

Williballenthin, C. (2017). Diving into the Windows Event Log file format. *DFIR Technical Journal*, 5(2), 44–57.

10. Anexos.

a. Código fuente de la herramienta.

```
"""
MAIN FILE
Provides:
- Permissions verification
- Program style definition
- Beggining of the file
"""

# -----
# Import libraries and modules
# -----
import sys
import ctypes
import os
from PyQt6.QtGui import QIcon
from PyQt6.QtWidgets import QApplication
from main_window import MainWindow

# Checks if the current process is running with administrator privileges on
Windows
def is_admin():
    try:
        return ctypes.windll.shell32.IsUserAnAdmin()
    except Exception:
        return False

# Restarts the application with administrator privileges (Windows only)
# Uses ShellExecuteW to request elevation and re-run the script with admin rights
def restart_as_admin():
    script = os.path.abspath(sys.argv[0])
    pythonw = sys.executable.replace("python.exe", "pythonw.exe")
    try:
        ctypes.windll.shell32.ShellExecuteW(None, "runas", pythonw, f'"{script}"',
        None, 1)
        sys.exit(0)
    except Exception as e:
        print("Elevation failed:", e)

# Loads and reads a QSS stylesheet file from the given file path
# Returns the stylesheet content as a string, or an empty string if the file is
not found
def load_stylesheet(path: str) -> str:
    try:
        with open(path, "r", encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        print(f"Stylesheet not valid")
        return ""

if __name__ == "__main__":
```

```

if not is_admin():
    restart_as_admin()
app = QApplication(sys.argv)
app.setWindowIcon(QIcon("icon.ico"))
style_path = os.path.join(os.path.dirname(__file__), "style.qss")
app.setStyleSheet(load_stylesheet(style_path))
win = MainWindow()
win.showMaximized()
sys.exit(app.exec())

```

```

"""
EVENT LOG READER FILE
Provides:
- Security copies of event files
- Parsing of events
# event_log_reader.py
"""

# -----
# Import libraries and modules
# -----
import os
import tempfile
import shutil

# Try pywin32
try:
    import win32evtlog
    import win32evtlogutil
    import win32con
except Exception:
    win32evtlog = None

# Try python-evtx
try:
    from Evtx.Evtx import Evtx
except Exception:
    Evtx = None

# Creates a temporary copy of an .evtx event log file to avoid issues with locked files
# Returns the path to the temporary copy
def safe_copy_evtx(path):
    """Copy file to temp to avoid locked-file issues."""
    dest_dir = tempfile.gettempdir()
    base = os.path.basename(path)
    dest = os.path.join(dest_dir, f"evtx_copy_{base}")
    shutil.copy2(path, dest)
    return dest

# Reads an .evtx event log file and extracts events up to maxEvents limit
# Returns a list of dictionaries containing raw XML representation of each event
# Requires python-evtx library to be installed
def read_evtx_summary(path, maxEvents=5000):
    """

```

```

    Read an .evtx file and return a list of dicts with key '__raw_xml' (xml
string).
    Requires python-evtx (pip install python-evtx).
    """
    rows = []
    if not os.path.exists(path):
        raise FileNotFoundError(f"{path} not found")
    if Evtx is None:
        raise RuntimeError("python-evtx is not installed. Install with: pip
install python-evtx")
    tmp = safe_copy_evtx(path)
    with Evtx(tmp) as log:
        for i, rec in enumerate(log.records()):
            xml = rec.xml()
            rows.append({"__raw_xml": xml})
            if i + 1 >= maxEvents:
                break
    return rows

# Wrapper class around win32evtlog library for reading Windows Event Log channels
# Provides methods to access local event log channels and extract event
information
# Requires pywin32 library to be installed
class EventLogReader:
    """
    Wrapper around win32evtlog to read channels (if pywin32 is installed).
    Returns list of dicts with keys: Source, EventID, TimeCreated, Level, Message,
Computer
    """
    def __init__(self):
        pass

    # Reads events from a Windows event log channel (e.g., 'System', 'Security',
'Application')
    # Extracts event details like Source, EventID, TimeCreated, Level, Message,
and Computer
    # Returns a list of dictionaries containing up to maxEvents records from the
channel
    def readChannel(self, channel, maxEvents=5000):
        if win32evtlog is None:
            raise RuntimeError("pywin32 (win32evtlog) is required to read
channels. Install pywin32.")
        server = None # local
        try:
            hand = win32evtlog.OpenEventLog(server, channel)
        except Exception:
            raise
        flags = win32evtlog.EVENTLOG_BACKWARDS_READ |
win32evtlog.EVENTLOG_SEQUENTIAL_READ
        records = []
        total = 0
        while True:
            events = win32evtlog.ReadEventLog(hand, flags, 0)
            if not events:
                break
            for ev in events:
                try:

```

```

        src = str(ev.SourceName) if hasattr(ev, 'SourceName') else ""
        eid = int(ev.EventID & 0xFFFF) if hasattr(ev, 'EventID') else ""

        time = ev.TimeGenerated.Format() if hasattr(ev,
'TimeGenerated') else ""
        level = getattr(ev, 'EventType', "")
        msg = ""
        try:
            msg = win32evtlogutil.SafeFormatMessage(ev, channel)
        except Exception:
            msg = str(getattr(ev, 'StringInserts', "") or "")
        rec = {
            "Source": src,
            "EventID": str(eid),
            "TimeCreated": time,
            "Level": str(level),
            "Message": msg,
            "Computer": getattr(ev, 'ComputerName', "")
        }
        except Exception:
            rec = {"Source": "", "EventID": "", "TimeCreated": "",
"Level": "", "Message": "", "Computer": ""}
        records.append(rec)
        total += 1
        if total >= maxEvents:
            break
    if total >= maxEvents:
        break
    try:
        win32evtlog.CloseEventLog(hand)
    except Exception:
        pass
    return records

```

```

"""
MAIN WINDOW FILE
Provides:
- Shapping of the main window
- Calls to program main functions
- Security copies of event files
- Loading and exporting
- Population of tables
- XML Tree view
- Filtering functions
"""

# -----
# Import libraries and modules
# -----
import os
import re
import tempfile
import shutil
from xml.etree import ElementTree as ET
from datetime import datetime

```

```

from PyQt6.QtCore import Qt, QObject, pyqtSignal, QThread
from PyQt6.QtWidgets import (
    QMainWindow, QWidget, QVBoxLayout, QHBoxLayout, QSplitter,
    QPushButton, QLineEdit, QComboBox, QLabel, QTableWidget,
    QTableWidgetItem, QFileDialog, QMessageBox, QTextEdit, QDialog,
    QDialogButtonBox, QHeaderView, QTreeWidget, QTreeWidgetItem,
    QSizePolicy, QMenu
)
from PyQt6.QtGui import QColor, QBrush, QAction, QIcon

# Import of the reader module to make it easier to use
try:
    import event_log_reader as elr_mod
except Exception:
    elr_mod = None

# python-evtx fallback for files running without it being installed
try:
    from Evtx.Evtx import Evtx
except Exception:
    Evtx = None

# -----
# Helpers
# -----
# Creates a temporary copy of a file to avoid issues with locked or in-use event
log files
# Returns the full path to the temporary copy
def safe_copy_to_temp(path):
    dest_dir = tempfile.gettempdir()
    base = os.path.basename(path)
    dest = os.path.join(dest_dir, f"evtx_copy_{base}")
    shutil.copy2(path, dest)
    return dest

# -----
# Reader verification, either function or module
# -----
# Worker class that runs on a separate thread to read event log files or channels
# Emits signals when finished or encounters errors to update the main UI safely
class ReaderWorker(QObject):
    finished = pyqtSignal(list)
    error = pyqtSignal(str)

    def __init__(self, path_or_channel, max_events=5000):
        super().__init__()
        self.path_or_channel = path_or_channel
        self.max_events = max_events

    # Executes the event log reading operation on a separate thread
    # Detects if input is a file path or channel name and reads accordingly
    # Emits finished signal with event records or error signal if something goes
wrong
    def run(self):
        try:
            p = self.path_or_channel
            # file path

```



```

        if isinstance(p, str) and p.lower().endswith(".evtx"):
            if elr_mod and hasattr(elr_mod, "read_evtx_summary"):
                rows = elr_mod.read_evtx_summary(p, self.max_events)
                self.finished.emit(rows)
                return
            if Evtx:
                rows = []
                tmp = safe_copy_to_temp(p)
                with Evtx(tmp) as log:
                    for i, r in enumerate(log.records()):
                        xml = r.xml()
                        rows.append({"__raw_xml": xml})
                        if i + 1 >= self.max_events:
                            break
                self.finished.emit(rows)
                return
            self.error.emit("No available file reader. Install python-evtx or
provide read_evtx_summary.")
            return
        else:
            # channel name -> map to System32\winevt\Logs\<channel>.evtx (3
main options)
            system_root = os.environ.get("SystemRoot", r"C:\Windows")
            candidate = os.path.join(system_root, "System32", "winevt",
"Logs", f"{p}.evtx")
            if not os.path.exists(candidate):
                self.error.emit(f"Channel file not found: {candidate}")
                return
            copied = safe_copy_to_temp(candidate)
            if elr_mod and hasattr(elr_mod, "read_evtx_summary"):
                rows = elr_mod.read_evtx_summary(copied, self.max_events)
                self.finished.emit(rows)
                return
            if Evtx:
                rows = []
                with Evtx(copied) as log:
                    for i, r in enumerate(log.records()):
                        xml = r.xml()
                        rows.append({"__raw_xml": xml})
                        if i + 1 >= self.max_events:
                            break
                self.finished.emit(rows)
                return
            self.error.emit("No available reader for channel files. Install
python-evtx or provide read_evtx_summary.")
            return
    except Exception as e:
        import traceback
        self.error.emit(traceback.format_exc())

# -----
# XML tree population
# -----
# Recursively converts an XML element and its children into a tree structure for
display
# Shows attributes, text content, and child elements with proper hierarchy
# Automatically expands items up to the specified expand_level

```

```

def add_xml_element_to_tree(tree_parent, element, level=0, expand_level=1):
    tag = element.tag.split("}")[-1] if "}" in element.tag else element.tag
    item = QTreeWidgetItem([f"<{tag}>"])
    # attributes
    for k, v in element.attrib.items():
        attr_item = QTreeWidgetItem([f'Attribute: {k} = "{v}"'])
        item.addChild(attr_item)
    # text
    text = (element.text or "").strip()
    if text:
        text_item = QTreeWidgetItem([f'Text: "{text}"'])
        item.addChild(text_item)
    # children
    for child in element:
        child_item = add_xml_element_to_tree(child, child, level + 1,
expand_level)
        item.addChild(child_item)
    if level <= expand_level:
        item.setExpanded(True)
    return item

# Parses XML text and displays it as a tree in the tree widget
# Handles XML parsing errors and removes xmlns attributes for cleaner display
def populate_tree_from_xml(tree_widget, xml_text, expand_level=1):
    tree_widget.clear()
    if not xml_text:
        return
    try:
        txt = re.sub(r'\sxmlns="[^"]+"', '', xml_text, count=1)
        root = ET.fromstring(txt)
    except Exception:
        try:
            root = ET.fromstring(xml_text)
        except Exception:
            tree_widget.addTopLevelItem(QTreeWidgetItem(["<unparsable XML>"]))
            return
    top = add_xml_element_to_tree(None, root, 0, expand_level)
    tree_widget.addTopLevelItem(top)
    tree_widget.expandItem(top)
    for i in range(top.childCount()):
        top.child(i).setExpanded(True)
    tree_widget.header().setSectionResizeMode(0,
QHeaderView.ResizeMode.ResizeToContents)

# Searches for matching text in the XML tree and highlights found items with
yellow background
# Expands parent items of matches for easy visibility
# Supports both plain text and regex pattern matching (better not to use, as I
wasnt able to optimize it well)
def highlight_tree_matches(tree_widget, term, use_regex=False):
    if not term:
        def clear(item):
            item.setBackground(0, QBrush())
            for i in range(item.childCount()):
                clear(item.child(i))
        for i in range(tree_widget.topLevelItemCount()):
            clear(tree_widget.topLevelItem(i))

```

```

        return
    try:
        pattern = re.compile(term, re.IGNORECASE) if use_regex else None
    except re.error:
        pattern = None
    def check_and_mark(item):
        text = item.text(0) or ""
        matched = False
        if use_regex and pattern:
            try:
                if pattern.search(text):
                    matched = True
            except re.error:
                matched = False
        else:
            if term.lower() in text.lower():
                matched = True
        if matched:
            item.setBackground(0, QBrush(QColor("#FFD44D")))
            parent = item.parent()
            while parent is not None:
                parent.setExpanded(True)
                parent = parent.parent()
        else:
            item.setBackground(0, QBrush())
        for i in range(item.childCount()):
            check_and_mark(item.child(i))
    for i in range(tree_widget.topLevelItemCount()):
        check_and_mark(tree_widget.topLevelItem(i))

# -----
# Modal FilterDialog for advanced filters
# -----
# Modal dialog that allows users to create complex filtering rules
# Each rule consists of a field, operator, and one or two values for range-based
operators
# Supports operators like =, !=, >, <, contains, regex, between, etc. (Better use
the normal ones, same problem with regex)
class FilterDialog(QDialog):
    def __init__(self, parent=None, get_fields_callable=None):
        # Modal configuration
        super().__init__(parent)
        self.setWindowTitle("Advanced filters")
        self.setModal(True)
        self.resize(640, 360)
        self.get_fields_callable = get_fields_callable
        self.setWindowIcon(QIcon("filters.ico"))

        # Headers section for filter (layout)
        layout = QVBoxLayout(self)
        self.rules_table = QTableWidget()
        self.rules_table.setColumnCount(4)
        self.rules_table.setHorizontalHeaderLabels(["Field", "Operator", "Value",
"Value2 (if between)"])

self.rules_table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.St
retch)

```

```

        layout.addWidget(self.rules_table)

        # Configuration of layout controls
        controls = QWidget()
        c_layout = QHBoxLayout()
        c_layout.setContentsMargins(0,0,0,0)
        controls.setLayout(c_layout)

        # Setting functionality buttons
        self.add_btn = QPushButton("+ Add Rule")
        self.apply_btn = QPushButton("Apply")
        self.clear_btn = QPushButton("Clear")
        self.cancel_btn = QPushButton("Cancel")

        # styling properties used by stylesheet in main.py
        self.apply_btn.setProperty('primary', True)
        self.clear_btn.setProperty('secondary', True)

        # Positioning of buttopns in modal view
        c_layout.addWidget(self.add_btn)
        c_layout.addStretch(1)
        c_layout.addWidget(self.apply_btn)
        c_layout.addWidget(self.clear_btn)
        c_layout.addWidget(self.cancel_btn)
        layout.addWidget(controls)

        # Connection between rules functionality and visual elements
        self.add_btn.clicked.connect(self.add_empty_rule)
        self.apply_btn.clicked.connect(self.on_apply)
        self.clear_btn.clicked.connect(self.on_clear)
        self.cancel_btn.clicked.connect(self.reject)
        self.rules = []      # rules counter list

    # Returns the list of available fields for filtering
    # Can use a custom callable or fall back to default field list
    def refresh_fields(self):
        if callable(self.get_fields_callable):
            return self.get_fields_callable()
        return ["Source", "EventID", "TimeCreated", "Level", "Message",
"Computer"]

    # Adds a new empty filter rule row to the table with dropdowns and text inputs
    def add_empty_rule(self):
        row = self.rules_table.rowCount()
        self.rules_table.insertRow(row)
        # Field combobox
        field_combo = QComboBox()
        field_combo.addItems(self.refresh_fields())
        self.rules_table.setCellWidget(row, 0, field_combo)
        # Operator
        op_combo = QComboBox()
        op_combo.addItems(["=", "!", ">", "<", ">=", "<=", "contains",
"startswith", "endswith", "regex", "between"])
        self.rules_table.setCellWidget(row, 1, op_combo)
        # Value inputs
        val_edit = QLineEdit()
        val_edit.setProperty('dialoginput', True)

```

```

        self.rules_table.setCellWidget(row, 2, val_edit)
        val2_edit = QLineEdit()
        val2_edit.setProperty('dialoginput', True)
        self.rules_table.setCellWidget(row, 3, val2_edit)

    # Extracts all filter rules from the dialog table and returns them as a list
    # of dictionaries
    # Skips empty rows and returns only complete rules with values
    def get_rules(self):
        rules = []
        for r in range(self.rules_table.rowCount()):
            fw = self.rules_table.cellWidget(r, 0)
            ow = self.rules_table.cellWidget(r, 1)
            vw = self.rules_table.cellWidget(r, 2)
            v2w = self.rules_table.cellWidget(r, 3)
            if not fw or not ow:
                continue
            field = fw.currentText()
            op = ow.currentText()
            val = vw.text().strip() if vw else ""
            val2 = v2w.text().strip() if v2w else ""
            # require at least a value for most ops
            if val == "" and op not in ("is empty", "is not empty"):
                continue
            rules.append({"field": field, "op": op, "value": val, "value2": val2})
        return rules

    # Collects all rules and closes the dialog with acceptance status
    def on_apply(self):
        self.rules = self.get_rules()
        self.accept()

    # Clears all filter rules from the table
    def on_clear(self):
        self.rules_table.setRowCount(0)
        self.rules = []

# -----
# Parse event XML to canonical dict (Source, EventID, TimeCreated, Level, Message,
# other EventData fields)
# -----
# Extracts event information from Windows Event Log XML format
# Parses System section for provider, event ID, time, level, and computer
# Also extracts Message and EventData fields for additional details
def parse_event_xml_to_dict(xml_text):
    if not xml_text:
        return {}
    try:
        xml2 = re.sub(r'\sxmlns="[^"]+"', '', xml_text, count=1)
        root = ET.fromstring(xml2)
    except Exception:
        try:
            root = ET.fromstring(xml_text)
        except Exception:
            return {}
    result = {}
    system = None

```

```

for child in root:
    if child.tag.lower().endswith("system"):
        system = child
        break
if system is None:
    for elem in root.iter():
        if elem.tag.lower().endswith("system"):
            system = elem
            break
if system is not None:
    for elem in system:
        lname = elem.tag.split('}')[1] if '}' in elem.tag else elem.tag
        if lname.lower() == "provider":
            name = elem.attrib.get("Name") or elem.text or ""
            if name:
                result["Source"] = name
        elif lname.lower() == "eventid":
            text = elem.text or ""
            if text:
                result["EventID"] = text.strip()
        elif lname.lower() == "timecreated":
            st = elem.attrib.get("SystemTime") or elem.text or ""
            if st:
                result["TimeCreated"] = st.strip()
        elif lname.lower() == "computer":
            text = elem.text or ""
            if text:
                result["Computer"] = text.strip()
        elif lname.lower() == "level":
            text = elem.text or ""
            if text:
                result["Level"] = text.strip()

# Message detection
msg = ""
for elem in root.iter():
    if elem.tag.split('}')[1].lower() == "renderinginfo":
        for sub in elem.iter():
            if sub.tag.split('}')[1].lower() == "message":
                msg = (sub.text or "").strip()
                break
        if msg:
            break
if not msg:
    for elem in root.iter():
        if elem.tag.split('}')[1].lower() == "message":
            msg = (elem.text or "").strip()
            break
eventdata = {}
for elem in root.iter():
    if elem.tag.split('}')[1].lower() == "eventdata":
        for d in elem:
            if d.tag.split('}')[1].lower() == "data":
                name = d.attrib.get("Name") or ""
                text = d.text or ""
                key = name if name else f"Data_{len(eventdata)+1}"
                eventdata[key] = text
        break

```

```

    if msg:
        result["Message"] = msg
    for k, v in eventdata.items():
        if k not in result:
            result[k] = v
    for k in ["Source", "EventID", "TimeCreated", "Level", "Message", "Computer"]:
        if k not in result:
            result[k] = ""
    return result

# -----
# Main window
# -----
# Main application window that displays event logs in a table with XML tree view
# Provides filtering, searching, export functionality, and event highlighting
class MainWindow(QMainWindow):
    # Initializes the main window UI with table, tree view, buttons, and all
    controls
    # Sets up signal/slot connections and state variables for data management
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Windows Event Log Analyzer")
        self.resize(1300, 850)

        # Top bar
        top = QWidget()
        top_l = QHBoxLayout()
        top_l.setContentsMargins(0,0,0,0)
        top_l.setSpacing(6)
        top.setLayout(top_l)
        top.setFixedHeight(42)
        top.setSizePolicy(QSizePolicy.Policy.Expanding, QSizePolicy.Policy.Fixed)

        # Controls (no quick filters; single Filters button)
        self.channel_combo = QComboBox()
        self.channel_combo.addItem("Application", "System", "Security"])
        self.search_edit = QLineEdit()
        self.search_edit.setPlaceholderText("Global search")
        self.search_scope = QComboBox()
        self.search_scope.addItem("All", "Table only", "XML only"])
        self.filters_btn = QPushButton("Filters")
        self.load_btn = QPushButton("Load Events")
        self.load_file_btn = QPushButton("Load From File")
        self.export_btn = QPushButton("Export CSV")
        self.status_label = QLabel("Ready")

        top_l.addWidget(self.channel_combo)
        top_l.addWidget(self.search_edit, 1)
        top_l.addWidget(self.search_scope)
        top_l.addWidget(self.filters_btn)
        top_l.addWidget(self.load_btn)
        top_l.addWidget(self.load_file_btn)
        top_l.addWidget(self.export_btn)
        top_l.addWidget(self.status_label)

        # Table (left)
        self.table = QTableWidget()

```

```

        self.table.setColumnCount(5)
        self.table.setHorizontalHeaderLabels(["Source", "Event ID", "Time",
"Level", "Message"])
        self.table.verticalHeader().setVisible(False)
        self.table.setAlternatingRowColors(True)
        self.table.setSelectionBehavior(self.table.SelectionBehavior.SelectRows)

self.table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Interact
ive)

        self.table.horizontalHeader().setStretchLastSection(True)

        # header context menu for column filters (simple)
        header = self.table.horizontalHeader()
        header.setContextMenuPolicy(Qt.ContextMenuPolicy.CustomContextMenu)
        header.customContextMenuRequested.connect(self.on_header_context_menu)

        # XML tree (right)
        self.xml_tree = QTreeWidget()
        self.xml_tree.setHeaderHidden(True)
        self.xml_tree.setColumnCount(1)
        self.xml_tree.setUniformRowHeights(True)

        splitter = QSplitter(Qt.Orientation.Horizontal)
        left_w = QWidget()
        left_l = QVBoxLayout()
        left_l.setContentsMargins(0,0,0,0)
        left_l.addWidget(self.table)
        left_w.setLayout(left_l)
        splitter.addWidget(left_w)
        splitter.addWidget(self.xml_tree)
        splitter.setStretchFactor(0, 3)
        splitter.setStretchFactor(1, 2)

        # Root layout
        root = QWidget()
        root_l = QVBoxLayout()
        root_l.setContentsMargins(6,6,6,6)
        root_l.setSpacing(8)
        root_l.addWidget(top)
        root_l.addWidget(splitter)
        root.setLayout(root_l)
        self.setCentralWidget(root)

        # signals
        self.load_btn.clicked.connect(self.load_channel)
        self.load_file_btn.clicked.connect(self.load_from_file)
        self.export_btn.clicked.connect(self.export_csv)
        self.search_edit.textChanged.connect(self.on_search_changed)
        self.filters_btn.clicked.connect(self.open_filters_dialog)
        self.table.cellClicked.connect(self.on_row_clicked)
        self.table.cellDoubleClicked.connect(self.on_row_double_clicked)

        # state
        self.records = []
        self.canonical_cols = ["Source", "EventID", "TimeCreated", "Level",
"Message", "Computer"]
        self.header_keys = ["Source", "EventID", "TimeCreated", "Level",

```



```

"Message"]

    # filters
    self.active_filters = {}          # column filters {col_index: (type_str,
value))}
    self.advanced_filters = []        # structured advanced filters (list of
dicts)

    # threading
    self.thread = None
    self.worker = None

    # suspicious event highlighting map (EventID -> color)
    self.suspicious_map = {
        "4624": QColor("#1a3b1a"),
        "4625": QColor("#3b391a"),
        "4688": QColor("#1a263b"),
        "7045": QColor("#3b1a1a"),
        "1102": QColor("#3b1a1a"),
    }

    # -----
    # Header context menu
    # -----
    # Shows a context menu when right-clicking on table column headers
    # Allows filtering by selected cell value or clearing column filters
    def on_header_context_menu(self, pos):
        header = self.table.horizontalHeader()
        logical_index = header.logicalIndexAt(pos)
        if logical_index < 0:
            return
        column_name = header.model().headerData(logical_index,
Qt.Orientation.Horizontal)
        menu = QMenu(self)
        act_filter_by_value = QAction(f"Filter by value from selection...", self)
        act_clear = QAction(f"Clear filter '{column_name}'", self)
        menu.addAction(act_filter_by_value)
        menu.addAction(act_clear)
        action = menu.exec(header.mapToGlobal(pos))
        if action == act_filter_by_value:
            sel = self.table.selectedRanges()
            if sel:
                rng = sel[0]
                r = rng.topRow()
                c = logical_index
                it = self.table.item(r, c)
                if it:
                    value = it.text()
                    self.active_filters[logical_index] = ("Contains", value)
                    self.apply_filters_and_search()
            elif action == act_clear:
                if logical_index in self.active_filters:
                    del self.active_filters[logical_index]
                    self.apply_filters_and_search()

    # -----
    # Open advanced Filters dialog (modal)

```

```

# -----
# Opens the advanced filter dialog and applies selected rules to filter the
event table
# Changes filter button color when filters are active
def open_filters_dialog(self):
    dlg = FilterDialog(self, get_fields_callable=self.get_available_fields)
    dlg.refresh_fields()
    # pre-populate
    for rule in self.advanced_filters:
        dlg.add_empty_rule()
        r = dlg.rules_table.rowCount() - 1
        fw = dlg.rules_table.cellWidget(r, 0)
        ow = dlg.rules_table.cellWidget(r, 1)
        vw = dlg.rules_table.cellWidget(r, 2)
        v2w = dlg.rules_table.cellWidget(r, 3)
        try:
            if fw:
                idx = fw.findText(rule.get('field'))
                if idx >= 0:
                    fw.setCurrentIndex(idx)
        except Exception:
            pass
        try:
            if ow:
                idx2 = ow.findText(rule.get('op'))
                if idx2 >= 0:
                    ow.setCurrentIndex(idx2)
        except Exception:
            pass
        if vw:
            vw.setText(rule.get('value', ''))
        if v2w:
            v2w.setText(rule.get('value2', ''))
    if dlg.exec() == QDialog.DialogCode.Accepted:
        self.advanced_filters = dlg.rules or []
        if self.advanced_filters:
            self.filters_btn.setStyleSheet('background-color: #154e7a; color:
white;')
        else:
            self.filters_btn.setStyleSheet('')
            self.apply_filters_and_search()

# Returns a list of all available fields from loaded records for use in filter
rules
# Prioritizes canonical columns and removes duplicates
def get_available_fields(self):
    fields = list(self.header_keys)
    for c in self.canonical_cols:
        if c not in fields:
            fields.insert(0, c)
    seen = set()
    res = []
    for f in fields:
        if f not in seen:
            seen.add(f)
            res.append(f)
    return res

```

```

# -----
# Search & filters orchestration
# -----
# Triggered when search text changes; applies all filters and search terms
def on_search_changed(self, txt):
    self.apply_filters_and_search()

# Main filtering and search orchestration method
# Applies advanced filters, column filters, and global search to show/hide
table rows
# Supports regex patterns with 're:' prefix and different search scopes (All,
Table only, XML only)
def apply_filters_and_search(self):
    def check_advanced(rec):
        for rule in self.advanced_filters:
            field = rule.get('field')
            op = rule.get('op')
            val = rule.get('value')
            val2 = rule.get('value2', '')
            field_val = rec.get(field, '')
            if not self.evaluate_rule(field_val, op, val, val2, field):
                return False
        return True

    def check_column_filters(row_idx):
        for col_idx, (ftype, fval) in self.active_filters.items():
            try:
                item = self.table.item(row_idx, col_idx)
                cell_text = item.text() if item else ''
            except Exception:
                cell_text = ''
            if not self._match_filter(cell_text, ftype, fval):
                return False
        return True

    search_text = self.search_edit.text().strip()
    use_regex = False
    regex_text = ''
    if search_text.startswith('re:'):
        use_regex = True
        regex_text = search_text[3:]
    scope = self.search_scope.currentText()

    for r in range(self.table.rowCount()):
        visible = True
        rec = self.records[r] if r < len(self.records) else {}
        # advanced
        if self.advanced_filters:
            if not check_advanced(rec):
                visible = False
        # column filters
        if visible and self.active_filters:
            if not check_column_filters(r):
                visible = False
        # global search
        if visible and search_text != '':

```

```

        table_ok = False
        xml_ok = False
        if scope in ('All', 'Table only'):
            for c in range(self.table.columnCount()):
                item = self.table.item(r, c)
                if item:
                    txt = item.text()
                    if use_regex:
                        try:
                            if re.search(regex_text, txt, re.IGNORECASE):
                                table_ok = True
                                break
                        except re.error:
                            pass
                    else:
                        if search_text.lower() in txt.lower():
                            table_ok = True
                            break
        if scope in ('All', 'XML only'):
            raw = self.records[r].get('__raw_xml', '')
            if use_regex:
                try:
                    if re.search(regex_text, raw, re.IGNORECASE):
                        xml_ok = True
                except re.error:
                    xml_ok = False
            else:
                if search_text.lower() in (raw or '').lower():
                    xml_ok = True
        if scope == 'Table only':
            visible = table_ok
        elif scope == 'XML only':
            visible = xml_ok
        else:
            visible = (table_ok or xml_ok)
        self.table.setRowHidden(r, not visible)

    self.apply_event_highlighting()

    # highlight tree for current selection if search targets XML
    cur = self.table.currentRow()
    if cur >= 0:
        raw = self.records[cur].get('__raw_xml', '')
        if search_text != '' and scope in ('All', 'XML only'):
            if use_regex:
                highlight_tree_matches(self.xml_tree,
                                     regex_text,
                                     use_regex=True)
            else:
                highlight_tree_matches(self.xml_tree,
                                     search_text,
                                     use_regex=False)
        else:
            highlight_tree_matches(self.xml_tree, '', use_regex=False)

    # Evaluates a single filter rule against a field value
    # Handles different operators (=, !=, >, <, contains, regex, between, etc.)
    # Supports special comparison types for numeric EventIDs and datetime fields
    def evaluate_rule(self, field_val, op, val, val2, field_name):

```



```

        return False
    else:
        return False
    # parse target(s)
    if op in ('>', '<', '>=', '<=', '=', '!='):
        try:
            v_dt = datetime.fromisoformat(val.replace('Z', '+00:00'))
        except Exception:
            try:
                v_dt = datetime.fromisoformat(val)
            except Exception:
                return False
        if op == '>':
            return fv_dt > v_dt
        if op == '<':
            return fv_dt < v_dt
        if op == '>=':
            return fv_dt >= v_dt
        if op == '<=':
            return fv_dt <= v_dt
        if op == '=':
            return fv_dt == v_dt
        if op == '!=':
            return fv_dt != v_dt
    if op == 'between':
        try:
            v1 = datetime.fromisoformat(val.replace('Z', '+00:00'))
            v2 = datetime.fromisoformat(val2.replace('Z', '+00:00'))
            return v1 <= fv_dt <= v2
        except Exception:
            return False
    if op == 'regex':
        try:
            return re.search(val, fv, re.IGNORECASE) is not None
        except re.error:
            return False
    if op == 'contains':
        return val.lower() in fv.lower()
    return False
except Exception:
    return False
else:
    # string comparisons
    if op == '=':
        return fv.lower() == val.lower()
    if op == '!=':
        return fv.lower() != val.lower()
    if op == 'contains':
        return val.lower() in fv.lower()
    if op == 'startswith':
        return fv.lower().startswith(val.lower())
    if op == 'endswith':
        return fv.lower().endswith(val.lower())
    if op == 'regex':
        try:
            return re.search(val, fv, re.IGNORECASE) is not None
        except re.error:

```

```

        return False
    # fallback numeric compare
    try:
        fv_n = float(fv)
        v_n = float(val)
        if op == '>':
            return fv_n > v_n
        if op == '<':
            return fv_n < v_n
        if op == '>=':
            return fv_n >= v_n
        if op == '<=':
            return fv_n <= v_n
    except Exception:
        pass
    return False

# Tests if a cell value matches a column filter of a given type (Contains,
Equals, Starts with, etc.)
def _match_filter(self, cell_text, ftype, fval):
    if cell_text is None:
        cell_text = ''
    if ftype == 'Contains':
        return fval.lower() in cell_text.lower()
    elif ftype == 'Equals':
        return cell_text.lower() == fval.lower()
    elif ftype == 'Starts with':
        return cell_text.lower().startswith(fval.lower())
    elif ftype == 'Ends with':
        return cell_text.lower().endswith(fval.lower())
    elif ftype == 'Regex':
        try:
            return re.search(fval, cell_text, re.IGNORECASE) is not None
        except re.error:
            return False
    return False

# -----
# Loading events (channel or file)
# -----
# Reads events from the selected Windows event log channel (Application,
System, Security)
def load_channel(self):
    channel = self.channel_combo.currentText()
    self._start_read(channel)

# Opens a file dialog to select an .evtx file and loads events from it
def load_from_file(self):
    default_dir = os.path.join(os.environ.get("SystemRoot", r"C:\Windows"),
"System32", "winevt", "Logs")
    path, _ = QFileDialog.getOpenFileName(self, "Select EVTX", default_dir,
"Event Log Files (*.evtx)")
    if not path:
        return
    self._start_read(path)

# Starts the event reading operation in a separate thread to avoid UI freezing

```

```

# Disables UI controls during reading and shows status
def _start_read(self, path_or_channel):
    self.load_btn.setEnabled(False)
    self.load_file_btn.setEnabled(False)
    self.export_btn.setEnabled(False)
    self._set_status("Reading...")
    self.thread = QThread()
    self.worker = ReaderWorker(path_or_channel)
    self.worker.moveToThread(self.thread)
    self.thread.started.connect(self.worker.run)
    self.worker.finished.connect(self._on_read_finished)
    self.worker.error.connect(self._on_read_error)
    self.worker.finished.connect(self.thread.quit)
    self.worker.finished.connect(self.worker.deleteLater)
    self.thread.finished.connect(self.thread.deleteLater)
    self.thread.start()

# Handles errors during event log reading and displays error message to user
def _on_read_error(self, msg):
    self._set_status("Error")
    QMessageBox.critical(self, "Read error", str(msg))
    self.load_btn.setEnabled(True)
    self.load_file_btn.setEnabled(True)
    self.export_btn.setEnabled(True)

# -----
# Read finished: parse XML and populate table
# -----
# Processes loaded event records, parses XML, and populates the table with
event data
# Dynamically determines columns based on union of all record fields
def _on_read_finished(self, records):
    self.records = records or []
    parsed_records = []
    for rec in self.records:
        raw = rec.get("__raw_xml", "")
        parsed = parse_event_xml_to_dict(raw)
        parsed["__raw_xml"] = raw
        parsed_records.append(parsed)
    self.records = parsed_records

# Build columns based on union of keys
keys_union = set()
for r in self.records:
    keys_union.update(k for k in r.keys() if k != "__raw_xml")
final_keys = []
for c in self.canonical_cols:
    for k in keys_union:
        if k.lower() == c.lower() and k not in final_keys:
            final_keys.append(k)
others = [k for k in sorted(keys_union) if k not in final_keys]
final_keys.extend(others)
if not final_keys:
    final_keys = ["Source", "EventID", "TimeCreated", "Level", "Message"]
display_keys = final_keys[:12]
self.header_keys = display_keys

```



```

# Populate table
self.table.setColumnCount(len(display_keys))
header_labels = []
for k in display_keys:
    lab = k
    lab = lab.replace("EventID", "Event ID").replace("TimeCreated",
"Time")
    header_labels.append(lab)
self.table.setHorizontalHeaderLabels(header_labels)
self.table.setRowCount(len(self.records))
for r_idx, rec in enumerate(self.records):
    for c_idx, key in enumerate(display_keys):
        v = rec.get(key, "")
        if v is None:
            v = ""
        if not isinstance(v, str):
            v = str(v)
        item = QTableWidgetItem(v)
        item.setToolTip(v)
        self.table.setItem(r_idx, c_idx, item)

self.table.resizeColumnsToContents()
self._set_status(f"Loaded {len(self.records)} records")
self.load_btn.setEnabled(True)
self.load_file_btn.setEnabled(True)
self.export_btn.setEnabled(True)
self.xml_tree.clear()
self.apply_event_highlighting()

# -----
# Row click / double click
# -----
def on_row_clicked(self, row, col):
    if 0 <= row < len(self.records):
        raw = self.records[row].get("__raw_xml", "")
        populate_tree_from_xml(self.xml_tree, raw, expand_level=1)
        search_text = self.search_edit.text().strip()
        if search_text.startswith("re:"):
            highlight_tree_matches(self.xml_tree, search_text[3:],
use_regex=True)
        elif search_text != "" and self.search_scope.currentText() in ("All",
"XML only"):
            highlight_tree_matches(self.xml_tree, search_text,
use_regex=False)
        else:
            highlight_tree_matches(self.xml_tree, "", use_regex=False)

# Opens a modal dialog showing the full formatted XML of the double-clicked
event for detailed inspection
def on_row_double_clicked(self, row, col):
    if 0 <= row < len(self.records):
        raw = self.records[row].get("__raw_xml", "")
        pretty = raw
        try:
            from xml.dom import minidom
            txt = re.sub(r'\sxmlns="[^"]+', '', raw, count=1)
            dom = minidom.parseString(txt)

```

```

        pretty = dom.toprettyxml(indent=" ")
        pretty = "\n".join([line for line in pretty.splitlines() if
line.strip() != ""])
    except Exception:
        pass
    dlg = QDialog(self)
    dlg.setWindowTitle("Event XML (detail)")
    dlg.setModal(True)
    v = QVBoxLayout(dlg)
    te = QTextEdit()
    te.setReadOnly(True)
    te.setPlainText(pretty)
    v.addWidget(te)
    btns = QDialogButtonBox(QDialogButtonBox.StandardButton.Close)
    v.addWidget(btns)
    btns.rejected.connect(dlg.reject)
    dlg.resize(900, 600)
    dlg.exec()

# -----
# Highlight suspicious events
# -----
# Applies background color highlighting to rows with suspicious/high-risk
event IDs
# Uses a predefined map of event ID to color for visual identification
def apply_event_highlighting(self):
    for r in range(self.table.rowCount()):
        hidden = self.table.isRowHidden(r)
        eid = ''
        for idx, key in enumerate(self.header_keys):
            if key.lower() in ('eventid', 'event id'):
                try:
                    eid = self.table.item(r, idx).text()
                except Exception:
                    eid = ''
                break
        eid_str = eid.strip()
        color = None
        if eid_str in self.suspicious_map:
            color = self.suspicious_map[eid_str]
        for c in range(self.table.columnCount()):
            it = self.table.item(r, c)
            if it is None:
                continue
            if color and not hidden:
                it.setBackground(QBrush(color))
            else:
                it.setBackground(QBrush())

# -----
# Utilities
# -----
# Updates the status label text in the main window
def _set_status(self, text):
    self.status_label.setText(text)

# Exports currently loaded and visible event records to a CSV file

```

```

# Handles special characters and newlines in field values for proper CSV
formatting (Just the simple one, the other was too weird)
def export_csv(self):
    if not self.records or not self.header_keys:
        QMessageBox.information(self, "No data", "No records to export.")
        return
    path, _ = QFileDialog.getSaveFileName(self, "Save CSV", "", "CSV Files
(*.csv)")
    if not path:
        return
    try:
        with open(path, "w", encoding="utf-8") as f:
            f.write(",".join(self.header_keys) + "\n")
            for rec in self.records:
                row = []
                for k in self.header_keys:
                    v = rec.get(k, "")
                    if v is None:
                        v = ""
                    if not isinstance(v, str):
                        v = str(v)
                    row.append(v.replace("\n", " ").replace(",", ";"))
                f.write(",".join(row) + "\n")
            QMessageBox.information(self, "Exported", f"Saved CSV to: {path}")
    except Exception as e:
        QMessageBox.critical(self, "Export error", str(e))

```

1. Listado de gráficos

Gráfico 1. Visión general de la arquitectura	28
Gráfico 2. Verificación de permisos de ejecución	30
Gráfico 3. Manejo de archivos temporales	30
Gráfico 4. Validación de dependencias	31
Gráfico 5. Ejemplo de manejo de errores	31
Gráfico 6. Propuesta de interfaz grafica	35
Gráfico 7. Mockup del programa	42
Gráfico 8. Interfaz propuesta	42
Gráfico 9. Flujo de datos.	43

2. Listado de tablas

Tabla 1. Funcionalidad de módulos	29
Tabla 2. Fases de desarrollo	29
Tabla 3. Modelo de conceptual	39
Tabla 4. Procesamiento del programa.	43